OWASP GenAI SECURITY PROJECT

GENAI SECURITY PROJECT

Securing Agentic Applications V2

A Builder & Defender's Guide

Version 2.0
October 2026

License and Usage

This document is licensed under Creative Commons, CC BY-SA 4.0.

You are free to:

- Share — copy and redistribute the material in any medium or format for any purpose, even commercially.
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

- Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.
- No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Link to full license text: https://creativecommons.org/licenses/by-sa/4.0/legalcode

The information provided in this document does not, and is not intended to constitute legal advice. All information is for general informational purposes only. This document contains links to other third-party websites. Such links are only for convenience and OWASP does not recommend or endorse the contents of the third-party sites.

Table of Content

1. Introduction 9
1.1 Purpose and Goals 9
1.2 Scope and Audience 9
1.3 Relationship with other ASI work 10
2. Secure Agentic Architectures 11
2.1 Defining Key Components of Agentic Architectures 11
2.2 Attack Surface Analysis 17
2.2.1 KC1 - Large Language Models (LLMs) 17
2.2.2 KC2 - Orchestration (Control Flow) 17
2.2.3 KC3 - Reasoning/Planning 17
2.2.4 KC4 - Memory Modules 17
2.2.5 KC5 - Tool Integration Frameworks 18
2.2.6 KC6 - Operational Environment 18
2.3 Secure Architecture Patterns 21
2.3.1 Core patterns being used in all frameworks 21
2.3.2 Sequential Agent Architecture 23
2.3.3 Hierarchical Agent Architecture 23

2.3.4 Collaborative Agent Swarm 23
2.4 Frameworks 25
2.4.1 General-Purpose Agent Frameworks 25
2.4.2 Enterprise and Platform-Integrated Frameworks 26
2.4.3 Data-Oriented Agent Frameworks 26
2.4.4 Specialized Agent Frameworks 26
2.5 Comprehensive Table: Key Components, Patterns, and Frameworks 27
2.6 Case Studies 28
2.6.1 Multi-agent co-pilot application 28
2.7 Cross-Cutting Architectural Concerns 30
2.7.1 The Agent Control Plane (instruction & policy inputs) 31
2.7.2 Agent Identity & Delegation 32
2.7.3 Capability Composition Risk (Lethal Trifecta / Rule of Two) 34
3. Agentic Developer Guidelines 35
Agentic AI Developer Security Guidelines: A Lifecycle Approach 35
3.1 Secure Design & Development Phase 35
3.1.1 Threat Modeling for Agentic Systems 35
3.1.2 System Prompt Engineering & Hardening 35
3.1.3 Secure Coding Practices 36
3.1.4 Content Moderation Integration (Design) 36
3.1.5 Human-in-the-Loop (HITL) Design 37
3.1.6 Memory Security Design 37
3.1.7 Input/Output Validation & Sanitization Design 38
3.1.8 Authorization and Authentication 40
3.2 Secure Build & Deployment Phase 41
3.2.1 Static Analysis & Code Scanning (SAST) 41

3.2.2 Dependency Vulnerability Scanning (SCA) 41
3.2.3 Environment Hardening & Sandboxing 41
3.2.4 Secure Configuration Management 42
3.2.5 Pre-deployment Testing (Fuzzing, Pen Testing) 43
3.3 Secure Operations & Runtime Phase 43
3.3.1 Continuous Monitoring & Anomaly Detection 43
3.3.2 Runtime Guardrails & Automated Moderation 44
3.3.3 Logging, Auditing & Traceability 46
3.3.4 Vulnerability Scanning (Runtime) 47
3.3.5 Incident Response Planning 47
3.3.6 AI Bot Mitigation and controls 47
3.3.7 Security Hooks (Enforcement Points) 49
3.3.8 Behavioral Circuit Breakers / Kill Switches 55
4. Enhanced Security Actions for Agentic AI Systems 58
4.1 Single-Agent Systems 58
4.1.1 Authentication & Authorization 58
4.1.2 Data Protection 59
4.1.3 Code Security 59
4.1.4 Monitoring & Incident Response 60
4.1.5 Prompt Security 60
4.2 Multi-Agent Systems with Central Orchestrator 61
4.2.1 Authentication & Authorization 61
4.2.2 Orchestrator Security 61
4.2.3 Inter-Agent Communication 63
4.2.4 Trust Boundaries 64
4.3 Multi-Agent Systems with Swarm Architecture 64
4.3.1 Authentication & Authorization 64
4.3.2 Decentralized Identity & Trust 64

4.3.3 Secure Peer-to-Peer Communication 65
4.4 Cross-Architecture Security Considerations 66
5. Key Operational Capabilities 67
5.0 Capability Control Matrix 67
5.1 KC6.1 – API Access 69
5.2 KC6.2 – Code Execution 70
5.3 KC6.3 – Database Execution (Queries and RAG access) 71
5.4 KC6.4 – Web Use 72
5.5 KC6.5 – Controlling PC Operations (Filesystem, OS Commands) 73
5.5.1 Coding & Computer-Use Agent Guardrails 73
5.6 KC6.6 – Operating Critical Systems (e.g., SCADA controls) 75
6. Agentic AI and the Supply Chain 76
6.1 Code Security 76
6.1.1 - Third party libraries & frameworks 76
6.1.2 - LLM/Agent generated code 76
6.2 Environments & Development 76
6.2.1 - LLM / Logic System 76
6.2.2 - Version Control & Code Management 76
6.2.3 - Permission management 77
6.3 Agent & Tool Discovery 77
6.3.1 - Agent Cards 77
6.3.2 - Local vs Remote Agent Registries 77
6.4 Tool Lifecycle & MCP-Specific Supply-Chain Controls 77
6.4.1 Standardized Provenance Record 77
6.4.2 - Tool onboarding & catalog governance 78

6.4.3 - Tool definition integrity (anti "rug-pull") 78
6.4.4 - Treat tool descriptions and responses as adversarial input 79
6.4.5 - Asset Access Control Matrix 79
7. Assuring Agentic Applications 81
7.0 Assurance as Lifecycle Validation 81
7.0.1 Assurance objective 81
7.0.2 Definitions used in this chapter 82
7.0.3 The assurance case 83
7.0.4 Design-time assurance scoping 84
7.0.5 Design-to-validation handoff and revalidation triggers 84
7.1 Red Teaming Agentic Applications 86
7.2 Behavioral Testing for Agentic Applications 89
7.2.1 Custom Benchmark Construction 90
Step-by-Step Guide: Selecting a Security Benchmark for Agentic AI Applications 93
Explainability Audits 95
8. Secure Agentic Deployments 96
9. Securing Agentic Applications across different runtimes & deployment Environments 99
9.1 Harden the Virtual Machine (Base Level) 99
9.2 Contain the Agentic Runtime 100
9.3 Secure the Agent's Memory, Tools, and Context 100
9.4 Observability + Forensics 101
9.5 Identity, Authentication, and Agent Authorization 101
9.6 (Optional) Cloud-Specific Hardening 101
9.7 Runtime Enforcement Additions 102

Acknowledgements 104

OWASP GenAI Security Project Sponsors 105

Project Supporters 106

1. Introduction

1.1 Purpose and Goals

This guide aims to provide practical and actionable guidance for designing, developing, and deploying secure agentic applications powered by large language models (LLMs). It complements the OWASP Agentic AI Threats and Mitigations (ASI T&M) documentFrameworks are great for proof of concept, but when implemented in production, they create more dependencies.

1.2 Scope and Audience

Audience: This guide is primarily intended for intermediate to advanced software developers, AI/ML engineers, security architects, security engineers, and technical leads involved in building, securing, or assessing workflow and agentic AI applications.

In Scope:

- Technical security controls and best practices for agentic systems.
- Secure architectural patterns specific to agentic AI.
- Mitigation strategies for common agentic threats (See Appendix).
- Guidance across the development lifecycle (design, build, deploy, operate).
- Considerations for key components like LLMs, orchestration, memory, tools, and operational environments.
- Examples of applying security principles in different agentic architectures (single-agent, multi-agent, etc.).

Out of Scope:

- High-level governance frameworks, organizational policies, or CISO-level strategic guidance (These will be addressed in a separate "Managing Agentic Risk Guide").
- Detailed implementation guides for specific vendor platforms or frameworks (though examples may be used illustratively).
- Ethical considerations beyond their direct impact on security implementation (e.g., bias mitigation strategies unrelated to security vulnerabilities).
- Legal and compliance requirements specific to jurisdictions or industries, although the principles discussed support compliance efforts.

The document does not cover governance and CISO-related guidelines, which will follow as a Managing Agentic Risk Guide.

1.3 Relationship with other ASI work

This document is part of the broader OWASP Agentic Security Initiative (ASI) and is designed to work along with other ASI resources. Specifically:

- **ASI Threats & Mitigations (T&M):** This guide provides practical implementation details and architectural context for the mitigations proposed in the T&M document. Where the T&M list identifies what threats exist and what mitigations are needed, this guide focuses on how to implement those mitigations technically. We reference the T-codes (e.g., T1, T2) from the T&M list throughout this document.
- **ASI Red Teaming Guide:** This guide provides the builder and defender perspective, outlining controls that red teams (following the Red Teaming Guide) would test. Understanding the defenses detailed here is crucial for effective red teaming simulation design.

Essentially, this guide translates the conceptual threats and mitigation principles from other ASI work into actionable architectural patterns and developer guidelines.

2. Secure Agentic Architectures

2.1 Defining Key Components of Agentic Architectures

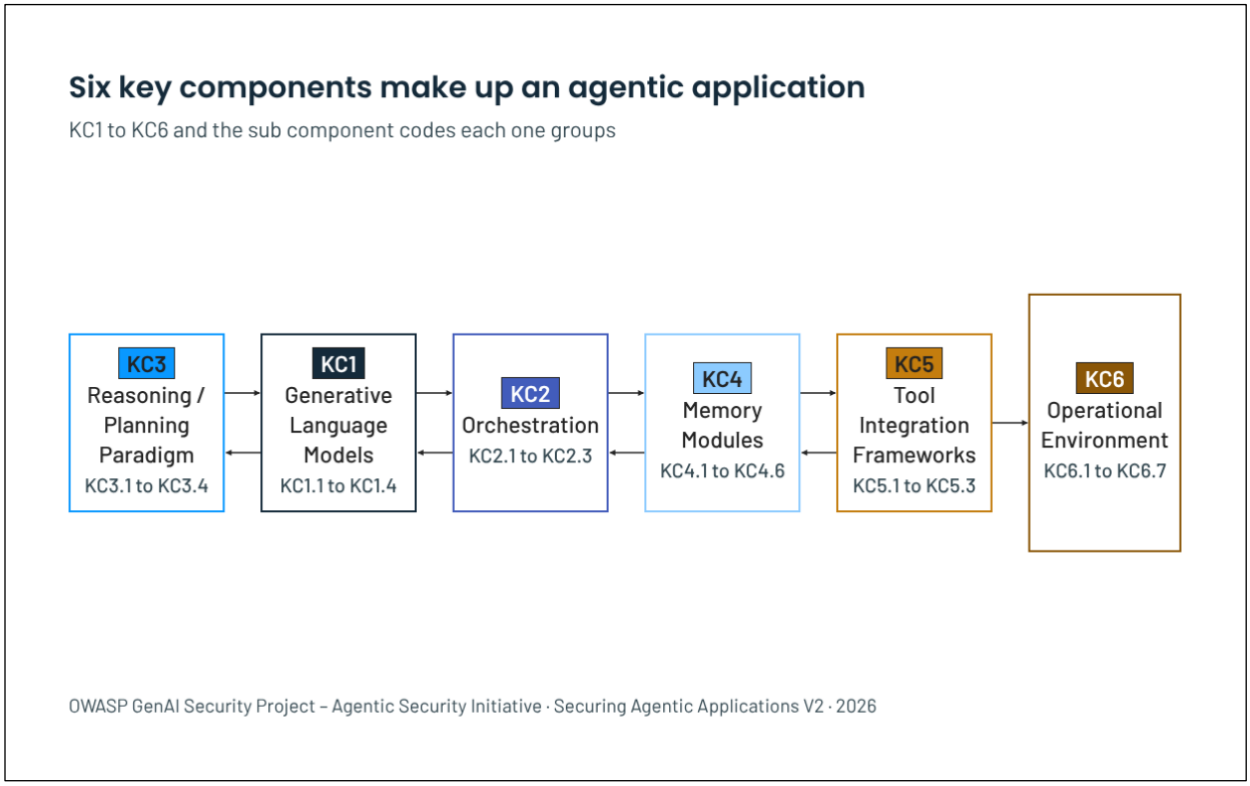


Figure 2.1: The six key components of an agentic application, KC1 Generative Language Models, KC2 Orchestration, KC3 Reasoning / Planning Paradigm, KC4 Memory Modules, KC5 Tool Integration Frameworks and KC6 Operational Environment, with the sub component code range each one groups. Each component keeps the same color in every later figure.

KC1 Generative Language Models: A probabilistic model component that interprets inputs and proposes content, plans, or actions; its output is untrusted input to deterministic orchestration, policy enforcement, and sink-side authorization. For hosted models, treat the provider as an external trust boundary and document prompt/response retention and secondary use, tenant isolation, processing and storage regions, subprocessors, where tools execute, model/version change notice, and drift monitoring.

- **KC1.1 Large Language Models (LLMs):** Probabilistic model components using pre-trained foundation models (e.g., GPT, Claude, Llama, or Gemini families) to propose reasoning artifacts, plans, and generated content under constraints such as context window, latency, and cost. Their outputs require deterministic validation and authorization before consequential use.
- **KC1.2 Multimodal LLMs (MLLMs):** LLMs capable of processing and/or generating information across multiple data types beyond text (e.g., images, audio), enabling agents to perform a wider variety of tasks involving visual or auditory information (e.g., GPT-4V, Gemini).
- **KC1.3 Small-Language Model (SLMs):** Language models with fewer parameters, which are trained on much smaller, more focused datasets. SLMs generally are designed to perform a specific task or work in a specific use case, rather than wielding the large, generalized capabilities found in an LLM. SLMs are characterized by a smaller weight space, parameter size, and context window when compared with LLMs.
- **KC1.4 Fine-tuned Models:** Language models (LLMs / MLLMs) that undergo additional training on specific datasets to specialize their capabilities, enhancing performance, adopting personas, or improving reliability for particular tasks, domains, or interaction styles required by the agent. For example, Supervised Fine Tuning (SFT) and Circuit Breakers (CB) have been found to bring promising scoping benefits to reduce the propensity to provide answers to out-of-scope topics.

KC2 Orchestration (Control Flow Mechanisms): Dictate the agent's overall behavior, information flow, and decision-making processes. The specific mechanism (e.g., "sequential" for layered architectures, "dynamic" for blackboard architectures, or "coordinated" for multi-agent systems) depends on the architecture and impacts the agent's responsiveness and efficiency.

- **KC2.1 Workflows:** A structured, pre-defined sequence of tasks or steps that agents follow to achieve a goal. Workflows define the flow of information and actions within the agent's operation and among agents. They can be linear, conditional, or iterative, depending on the complexity of the task.
- **KC2.2 Hierarchical Planning:** Multiple agents collaborating with each other via an orchestrator (router). The orchestrator plays a central role in this process. It is responsible for:
    - Understanding the Task: The orchestrator receives the initial complex task or request.
    - Decomposing the Task: The orchestrator then analyzes the task and breaks it down into a series of sub-tasks.
    - Routing to Specialized Agents: The orchestrator identifies which specialized agent is best suited to handle each sub-task and assigns the sub-task accordingly.
    - Orchestrator monitors agent performance, identifies inefficiencies, and autonomously adjusts workflows to optimize efficiency.
    - The Orchestrator can interface with the user directly or to a "master" agent that helps coordinate the various agents.

- **KC2.3 Multi-agent Collaboration:** This involves multiple agents working together to achieve a common goal. Agents can communicate and coordinate their actions, sharing information and resources. This approach is useful for complex tasks that require diverse skills or knowledge. For example, one agent might be responsible for data collection, while another is responsible for analysis, and a third is responsible for decision-making.

KC3 Reasoning / Planning Paradigm: Agents utilize LLMs to solve complex tasks requiring multiple steps using strategic thinking. To do so, agents break down high-level tasks into smaller tasks (steps), each with a different sub-goal. Enable AI agents to solve complex problems requiring multiple steps and logical thinking.

- **KC3.1 Structured Planning / Execution** (e.g., ReWoo, LLM Compiler, Plan-and-Execute), focuses on:
  - Decomposing tasks into a formal plan.
  - Defining sequences of actions, often involving specific tool calls.
  - Executing the plan, sometimes using separate "planner" and "executor" components.
- **KC3.2 ReAct (Reason + Act):** Dynamically interleaves reasoning steps with actions (like using tools or querying APIs) and updates reasoning based on feedback.
- **KC3.3 Chain of Thought (CoT):** Enhances reasoning quality by prompting step-by-step "thinking", inducing an LLM to generate a set of "thoughts" before arriving at a final action or conclusion.
- **KC3.4 Tree of Thoughts (ToT):** Generalizes Chain of ThoughtCoT by exploring multiple reasoning paths and plans in parallel with lookahead, backtracking, and self-evaluation.

KC4 Memory Modules: Enable the agent to retain short-term (immediate context) and long-term information (past interactions, knowledge) for coherent and personalized interactions. Context "sensitivity" (classification or compartmentalization) is used to reduce risk of unauthorized information exposure. Retrieval-Augmented Generation (RAG) used with a vector database is common for long-term memory, allowing agents to retrieve and incorporate external knowledge using semantic search.

- **KC4.1 In agent session memory:** Memory is limited to a single agent and a single session, limiting the ability to compromise additional agents/sessions if compromised.
- **KC4.2 Cross-agent session memory:** Memory is shared across multiple agents, but is limited to a single session, limiting the ability to compromise additional sessions, but a single compromised agent might cause compromising multiple agents operating at the same session.
- **KC4.3 In agent cross-session memory:** Memory is limited to a single agent, but is shared across multiple sessions, limiting the ability to compromise additional agents, but one compromised session might still cause compromising multiple sessions.
- **KC4.4 Cross-agent cross-session memory:** Memory is shared across multiple agents, and sessions. If an agent is compromised in a specific session, it might compromise additional sessions and/or agents.

- **KC4.5 In agent cross-user memory:** Memory is limited to a single agent, but is shared across multiple users. A compromised agent might have the ability to compromise multiple users.
- **KC4.6 Cross-agent cross-user:** Memory is shared across multiple agents, and users. A compromised agent might have the ability to compromise additional agents and users.

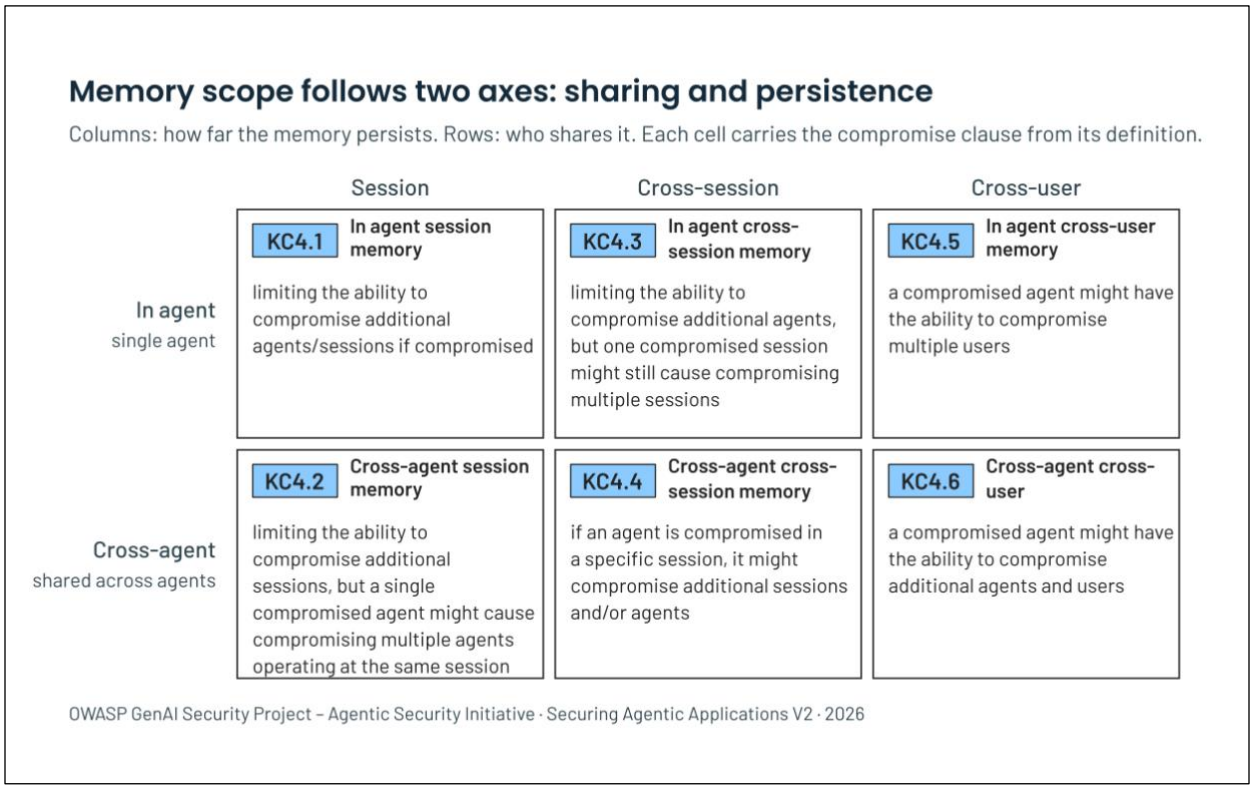


Figure 2.2: The six KC4 memory types laid out by who shares the memory (in agent or cross-agent) and how far it persists (session, cross-session, cross-user). Each cell carries the compromise clause from its definition.

KC5 Tool Integration Frameworks: Allow agents to extend their capabilities beyond text by using external tools (APIs, functions, data stores) to interact with the real world or other systems. Tool integration framework is used to manage the selection and use of these tools. These frameworks offer various "Agent" types, for example ReAct, Self-Ask, OpenAI Functions Agent. (For more information on specific protocols, see Appendix E).

- **KC5.1 Flexible Libraries / SDK Features:** These provide code-level building blocks (e.g. LangChain, AG2, Agents, CrewAI, MCP) or specific API capabilities (like OpenAI's Tool Use feature) for developers. They offer high flexibility and control over agent behavior and tool orchestration but require more coding effort and technical expertise to implement and manage the full agent loop.

- **KC5.2 Managed Platforms / Services:** These are vendor-provided solutions (like Amazon Bedrock Agents, Microsoft Copilot Platform) that handle infrastructure, simplify setup, and manage much of the orchestration for building agents with tools. They often feature easier integration within the provider's ecosystem and may include low-code interfaces, trading some flexibility for faster development and deployment.
- **KC5.3 Managed APIs:** Similar to managed platforms but often emphasizing the API as the main interaction point, these are vendor-hosted services (like OpenAI's Assistants API) providing higher-level abstractions. They manage complexities like state and aspects of tool orchestration via API calls, making it easier to build sophisticated, stateful agents compared to using basic SDK features alone.

KC6 Operational Environment (Agencies): Though Large Language Models (LLMs) are restricted to the data available up to their last training update, agents can interface with external environments through tools and function calls. Utilizing different agencies enables interaction with, gather, and process information from external environments, allowing them to operate effectively within them.

- **KC6.1 API Access:**
  - **KC6.1.1 Limited API Access:** An agent utilizing LLM capabilities to generate some of the parameters to a predefined API call (e.g., an agent utilizing LLM to generate a parameter used in a predefined REST/GraphQL API call). A compromised agent might have the ability to launch attacks on the API (see OWASP Top 10 for API), via the params generated by the LLM, resulting in parameter pollution attack.
  - **KC6.1.2 Extensive API Access:** An agent utilizing LLM capabilities to generate the entire API call (e.g., an agent utilizing LLM to interactively traverse a GraphQL API or to generate the entire URL to a REST API). A compromised agent might have the ability to generate unwanted API calls on behalf of the user by utilizing the agent's excessive authorization, and to launch attacks on the API (see OWASP Top 10 for API).
- **KC6.2 Code Execution:**
  - **KC6.2.1 Limited Code Execution Capability:** An agent utilizing LLM capabilities to generate some of the parameters to a predefined function (e.g., an agent utilizing LLM to generate a parameter used in a predefined python/php/js function). A compromised agent might have the ability to launch code injection attacks (see OWASP code injection), via the params generated by the LLM.
  - **KC6.2.2 Extensive Code Execution Capability:** An agent running code generated by an LLM. A compromised agent might have the ability to run arbitrary code.
- **KC6.3 Database Execution:**
  - **KC6.3.1 Limited Database Execution Capability:** An agent utilizing LLM capabilities to run specific queries or commands against a database. Limited database execution is characterized by an agent having a limited permission set, such as read only permissions at



the table or row level, and/or constructing write with access to only change certain parameters, using parameterization or pre-constructed queries. A compromised agent could exfiltrate a table from a database or write malformed / malicious data to a limited space in a database / table.

  - **KC6.3.2 Extensive Database Execution Capability:** An agent utilizing LLM capabilities to generate and run all CRUD operations against a set of tables or complete database. A compromised agent might have the ability to alter any record in the database, delete a database / table, or access and leak any information stored within the database.
  - **KC6.3.3 Agent Memory or Context Data Sources (e.g. RAG agent):** An agent utilizing external datasource to gain contextual information, or updates records in external sources based on user inputs. A compromised agent might disrupt data in the datasource, or provide malformed information retrieved from the datasource.
- **KC6.4 Web Access Capabilities (web-use):** An agent utilizing LLM for operating with the browser (e.g., OpenAI operator, browser-use). A compromised agent is likely to form due to exposure to untrusted web content, and it might perform unwanted operations on behalf of the user by utilizing the agent's excessive authorization (e.g., changing sharing settings or account settings on open user sessions).
- **KC6.5 Controlling PC Operations (PC-use):** An agent utilizing LLM for operating with the operation system, including file system (e.g., Anthropic computer use). A compromised agent might perform unwanted operations, expose or leak personal information and perform malicious operations such as encrypting files.
- **KC6.6 Operating Critical Systems (e.g., SCADA):** An agent utilizing LLM to operate critical system. Unauthorized operations in critical systems. A compromised agent can cause catastrophic failure in system operations.
- **KC6.7 Access to IoT Devices:** Unauthorized control over IoT devices. A compromised agent could impact the operational environment hosting such IoT devices, potentially utilizing them in malicious or unintended ways (e.g. drastically changing room temperature using an IoT connected thermostat).

2.2 Attack Surface Analysis

2.2.1 KC1 - Large Language Models (LLMs)

- **T5 (Cascading Hallucination):** Foundation models generate incorrect information that propagates
- **T6 (Intent Breaking):** Attacks target the core decision-making capabilities
- **T7 (Misaligned Behaviors):** Model alignment issues lead to unintended behaviors impacting users, organizations, or broader population
- **T15 (Human Manipulation):** Models exploit human trust to manipulate users

2.2.2 KC2 - Orchestration (Control Flow)

- **T6 (Intent Breaking):** Manipulates control flow to achieve unauthorized goals
- **T8 (Repudiation):** Makes agent actions difficult to trace through workflow
- **T9 (Identity Spoofing):** Especially problematic in multi-agent systems (KC2.2, KC2.3)
- **T10 (Overwhelming HITL):** Overwhelms human oversight in workflows
- **T12 (Communication Poisoning):** Corrupts inter-agent messaging in multi-agent systems
- **T13 (Rogue Agents):** Compromises agent orchestration in multi-agent systems
- **T14 (Human Attacks):** Exploits trust relationships between agents and workflows

2.2.3 KC3 - Reasoning/Planning

- **T5 (Cascading Hallucination):** Affects reasoning quality and propagates through planning steps
- **T6 (Intent Breaking):** Directly attacks the reasoning process to manipulate goals
- **T7 (Misaligned Behaviors):** Creates subtle reasoning misalignments
- **T8 (Repudiation):** Obscures decision trails in reasoning chains
- **T15 (Human Manipulation):** Leverages reasoning to craft manipulative responses

2.2.4 KC4 - Memory Modules

- **T1 (Memory Poisoning):** Directly targets all memory types (KC4.1-KC4.6)
- **T3 (Privilege Compromise):** Breaks information system boundary through context collapse, causing unauthorized data access/leakage (e.g. across tools)
- **T5 (Cascading Hallucination):** Stores and amplifies hallucinations across sessions or agents
- **T6 (Intent Breaking & Goal Manipulation):** Abuses shared context, breaking integrity, leaking/contamination, or interfering with data records/assets (e.g. within a tool) that should be isolated from one another
- **T8 (Repudiation):** Manipulates or erases evidence from memory
- **T12 (Communication Poisoning):** Affects shared memory in multi-agent systems

2.2.5 KC5 - Tool Integration Frameworks

- **T2 (Tool Misuse):** Core vulnerability for all tool integration types (via MCP as well). Unverified, untrusted, or compromised tools
- **T3 (Privilege Compromise):** Tools often run with specific privileges that can be exploited/excessive agency.
- **T6 (Intent Breaking & Goal Manipulation):** interfering with data records/assets (e.g. within a tool) that should be isolated from one another
- **T7 (Misaligned Behaviors):** Creates subtle reasoning misalignments
- **T8 (Repudiation):** Tool use may lack proper logging and auditability
- **T11 (Unexpected RCE):** Tools might enable unexpected code execution

2.2.6 KC6 - Operational Environment

- **T2 (Tool Misuse):** All operational environments can be misused, introducing more untrusted content and opportunity for agent failure, e.g. unauthorized settings changes.
- **T3 (Privilege Compromise):** Highest risk in environments with extensive access (KC6.1.2, KC6.2.2, KC6.3.2)
- **T4 (Resource Overload):** External services can be overwhelmed
- **T10 (Overwhelming HITL):** Creates excessive activity requiring human approval
- **T11 (Unexpected RCE):** Direct risk to code execution environments (KC6.2)
- **T12 (Agent communication poisoning):** Using external systems for side channel communications and memory persistence
- **T13 (Rogue agents):** Compromised AI agent activity outside monitoring limits
- **T15 (Human Manipulation):** Leverages operational access to manipulate humans

Summary table

Key Component	Associated Threats	Risk Description
KC1 - Large Language Models (LLMs)	T5, T6, T7, T15	Core vulnerabilities in the "brain" of the agent: hallucinations, goal misalignment, deceptive reasoning, and manipulating humans
KC2 - Orchestration (Control Flow)	T6, T8, T9, T10, T12, T13, T14	Vulnerabilities in workflow control: goal manipulation, lack of auditability, identity confusion, overwhelming human oversight, and multi-agent attacks
KC3 - Reasoning/Planning Paradigm	T5, T6, T7, T8, T15	Weaknesses in decision processes: cascading hallucinations, intent manipulation, deceptive reasoning, untraceable decisions, and human manipulation
KC4 - Memory Modules	T1, T3, T5, T6, T8, T12	Data integrity issues: memory poisoning, leaking, propagating hallucinations, evidence tampering, and communication poisoning
KC5 - Tool Integration Frameworks	T2, T3, T6, T8, T11	Tool exploitation vulnerabilities: tool misuse, privilege escalation, unlogged actions, and unexpected code execution
KC6 - Operational Environment	T2, T3, T4, T10, T11, T12, T13, T15	Monitoring gaps and External system risks: misusing access, privilege compromise, resource exhaustion, overwhelming human reviewers, code execution attacks, side channel information storage/communication, and manipulation via environment

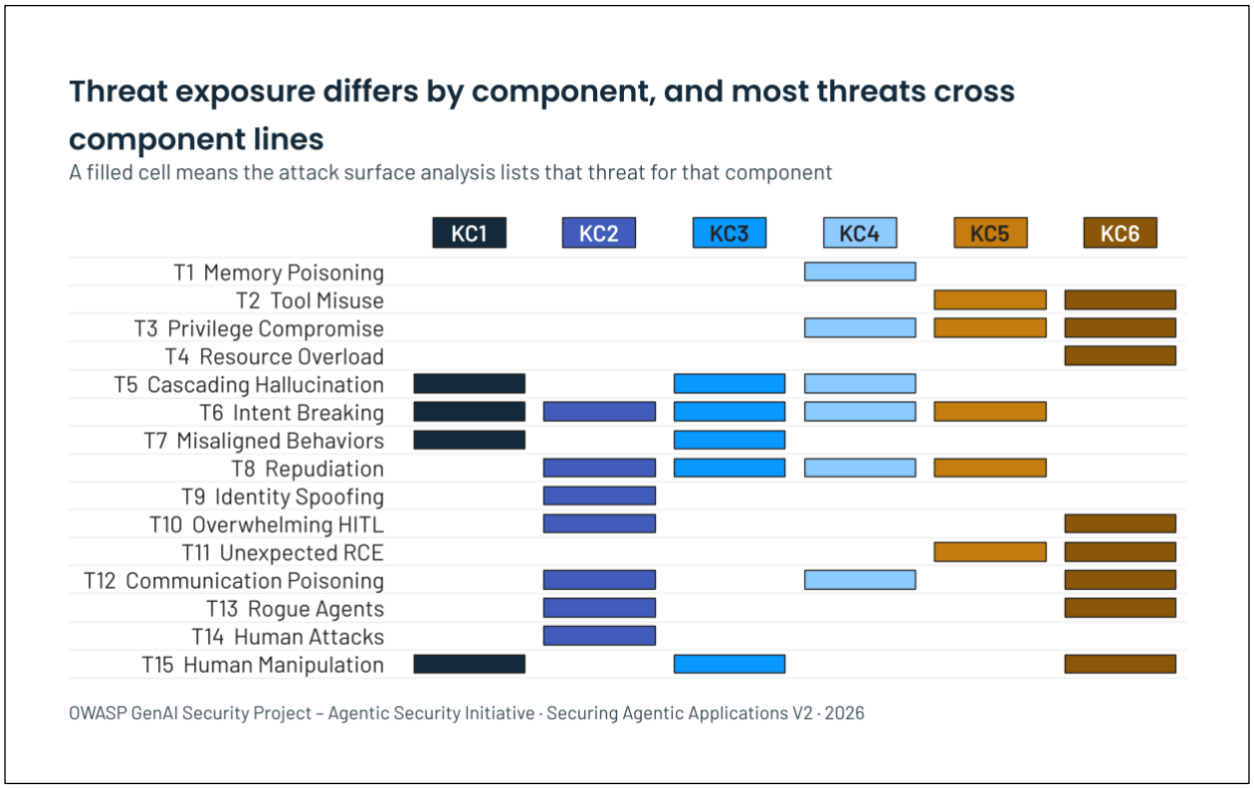


Figure 2.3: The threats T1 to T15 that the attack surface analysis associates with each key component. A filled cell means the summary table lists that threat for that component, in the component's color.

2.3 Secure Architecture Patterns

Building secure agentic systems requires more than just securing individual components; it demands a holistic approach where security is embedded within the architecture itself. The choice of architecture (e.g., sequential, hierarchical, swarm) significantly influences the attack surface and the effectiveness of different security controls. This section details fundamental security patterns applicable across various architectures and then explores how specific architectural choices impact the implementation of defenses like isolation, least privilege, and secure communication. By understanding these patterns and practices, architects and developers can design systems that are resilient to agentic-specific threats from the ground up. While core patterns act as the foundational building blocks for agentic systems, the architectures described below function as the structural blueprints that organize these blocks into cohesive frameworks.

2.3.1 Core patterns being used in all frameworks

While the landscape of agentic frameworks (like LangChain, CrewAI, AutoGPT, etc.) is diverse, several fundamental architectural patterns appear consistently. These patterns represent common solutions to core challenges in agent development, such as extending capabilities, improving reasoning, and enabling self-correction. Understanding these core patterns is crucial for security, as they often introduce specific control points and potential vulnerabilities regardless of the overarching framework used. The following patterns are frequently employed building blocks within larger agentic systems:

Tool Use Pattern:

- **Concept:** Equips agents with the ability to use external tools, APIs, or functions to extend their capabilities beyond internal knowledge or LLM reasoning. The agent delegates tasks like data retrieval, calculations, or actions to appropriate tools.
- **Key Components:** Requires tool definitions, an agent reasoning loop capable of selecting and invoking tools, and integration mechanisms.

Reflection Pattern:

- **Concept:** Enables agents to introspect, evaluate their past actions, decisions, or outputs against goals or metrics, and use this self-assessment to refine future strategies or correct errors.
- **Key Components:** Involves feedback loops, evaluation logic, and mechanisms for the agent to modify its own behavior or knowledge.

Retrieval Augmented Generation (RAG) Pattern:

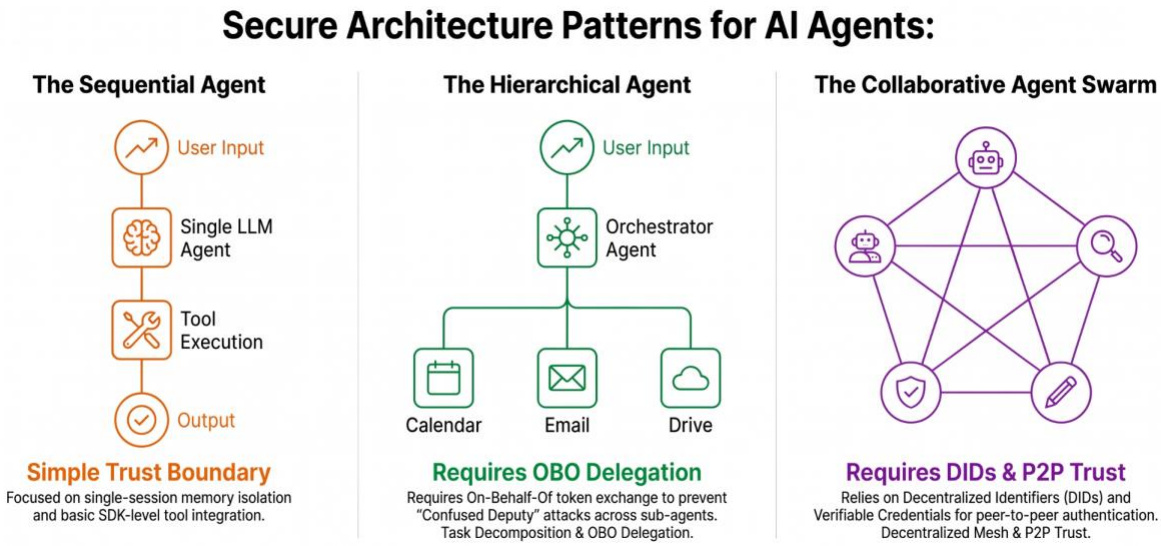
- **Concept:** Enhances LLM-based agents by first retrieving relevant information from an external knowledge base (vector store, database, etc.) based on the user query, and then providing this retrieved context to the LLM along with the original query to generate a more informed and accurate response.
- **Key Components:** Requires a retriever module, a knowledge base/vector store, and integration with the LLM prompt.

User privilege/auth assumption pattern for execution

- **Concept 1:** When an agent is executing against an API, database or entity which has a concept of granular user permissions, it should assume the permission of the user which invoked the agent and even scoped further by the task, resource, context or action allowed. This will enforce the granular security controls on the agent, preventing it from returning information the user shouldn't have access to, such as other user data or data which is not relevant to the current action. This may also be used in the Just-In-Time access and ephemeral access use cases.
- Additionally, if the user privileges are not already restricted by a strong "Chinese Wall" model (to prevent information in records leaking to unrelated records), the agent should track the boundary of the context it is operating on.
- **Key components:** LLM brain (KC1), Execution system for an API (KC6.1, KC6.3, KC6.4) and an authentication/authorization/IAM system.

Credential handling for browser/computer use

- **Concept:** When an AI agent performs browser or computer-use tasks that require authentication, route the user through a trusted user-mediated identity-provider, credential-broker, or password-manager flow. The model and agent context must never receive passwords, reusable session secrets, refresh tokens, or raw access tokens. An out-of-model executor may exercise an opaque handle or a short-lived, least-privilege, audience- and task-scoped credential only after explicit user authorization.



2.3.2 Sequential Agent Architecture

A straightforward linear workflow where a single agent processes input through planning, execution, and basic tool use. This pattern focuses on simplicity with a clear chain of thought and limited memory. This basic pattern uses a single LLM as the cognitive core with a linear workflow:

- **Brain:** Single LLM (KC1.1)
- **Control:** Simple sequential workflow (KC2.1)
- **Reasoning:** Chain of Thought reasoning (KC3.3)
- **Memory:** In-agent session memory only (KC4.1)
- **Tools:** Limited SDK integration (KC5.1)
- **Environment:** Restricted API access (KC6.1.1)

2.3.3 Hierarchical Agent Architecture

An orchestrator agent breaks down complex tasks and distributes them to specialized sub-agents. Each agent handles a specific domain using appropriate tools, with the orchestrator managing the overall process and integrating results. A more complex pattern with specialized sub-agents coordinated by an orchestrator:

- **Brain:** Multiple specialized LLMs / fine tuned models (KC1.1 / KC1.2)
- **Control:** Hierarchical planning with orchestrator (KC2.2)
- **Reasoning:** Structured planning / execution (KC3.1)
- **Memory:** Cross-agent session memory (KC4.2)
- **Tools:** Managed platforms / services (KC5.2)
- **Environment:** More extensive capabilities (KC6.2, KC6.4)

2.3.4 Collaborative Agent Swarm

A pattern where multiple peer agents work together without strict hierarchy:

- **Brain:** Multiple specialized agents (KC1.1 / KC1.2)
- **Control:** Multi-agent collaboration (KC2.3)
- **Reasoning:** Tree of Thoughts for exploring multiple solutions (KC3.4)
- **Memory:** Cross-agent cross-session memory (KC4.4)
- **Tools:** Flexible libraries with collaboration features (KC5.1)
- **Environment:** Distributed operational capabilities (various KC6)

These patterns can be mixed and matched based on specific application requirements, with security considerations becoming increasingly important as the agent's operational capabilities expand.

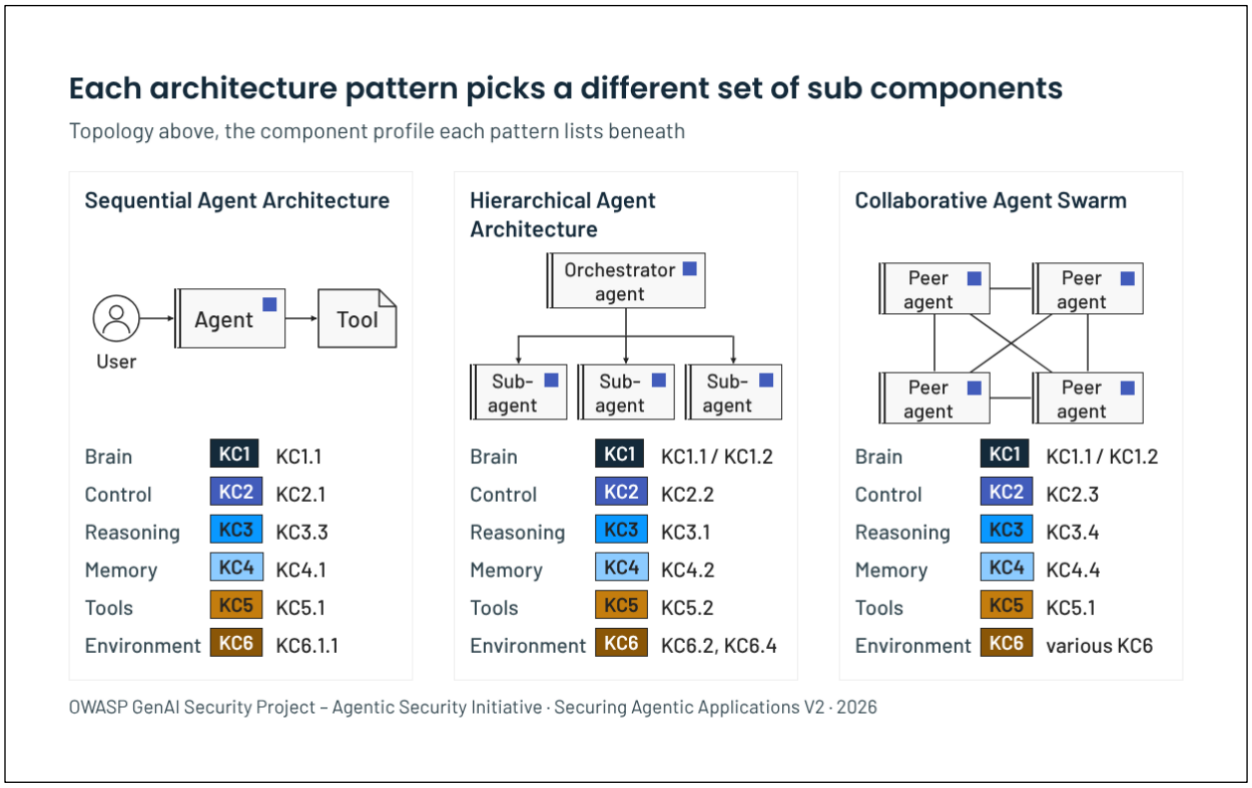


Figure 2.4: The Sequential Agent Architecture, Hierarchical Agent Architecture and Collaborative Agent Swarm side by side. Each panel shows the topology above and, beneath it, the Brain, Control, Reasoning, Memory, Tools and Environment profile the pattern lists, by sub component code.

2.4 Frameworks

Numerous frameworks and libraries have emerged to accelerate the development of agentic applications. These frameworks provide pre-built components, abstractions, and patterns (like those discussed in 2.2.1) to handle common tasks such as managing state, integrating tools, orchestrating workflows, and interacting with LLMs. However, the choice of framework can have significant security implications, influencing the default security posture, the ease of implementing controls, and the potential attack surface. Some frameworks are general-purpose, while others are specialized for specific tasks like multi-agent collaboration or data-intensive RAG. This section provides an overview of some popular frameworks, categorized by their primary focus.

2.4.1 General-Purpose Agent Frameworks

Caution: Customization and abstraction are a trade-off. Frameworks are great for proof of concept, but implementing them in production may create more dependencies. If you modify a framework to fit your unique needs, then later when that framework is patched due to a vulnerability being discovered, you're left needing a rapid update that could have significant impacts on your system because your customer code no longer fits the updated core. There is fragility in those types of situations that should be considered by the development team.

- **LangChain:** One of the most popular frameworks that provides components for building LLM-powered applications with memory systems, tool use, and various agent types like ReAct and Self-Ask. (See Appendix 1: LangChain best practices.)
- **AutoGPT:** An early autonomous agent framework that can plan and execute tasks with minimal human intervention, using a suite of tools and web access.
- **MetaGPT:** Focuses on multi-agent collaboration where different agents take on specialized roles (like product manager, engineer, tester) to solve complex problems.
- **CrewAI:** Specifically designed for collaborative agent systems where multiple specialized agents work together on tasks through defined roles and workflows.
- **AutoGen:** Microsoft Open Source Framework for building multi-agent systems. Each agent has a role, goal, memory and toolset.
- **OpenDevin:** Multimodal, agentic AI framework focused on DevOps and software engineering tasks. Uses a reasoning loop of reason + act + observe. Tools include a code editor, a browser, command line interface (CLI), execution environment, and Git version control.
- **LangGraph:** Python framework built on LangChain. Developer defines LLM agents as graphs (cyclical, asynchronous, and stateful workflows). Supports HITL (Human in the Loop) pause / resume checkpoints.
- **Dify:** Open Source LLM app development platform. It combines agentic AI workflow, RAG pipeline, agent capabilities, model management, and observability elements.

2.4.2 Enterprise and Platform-Integrated Frameworks

- **Amazon Bedrock Agents:** A managed service for building, testing and deploying AI agents with integration into AWS services and knowledge bases.
- **Microsoft Semantic Kernel:** An SDK that integrates AI with programming languages, allowing developers to create AI agents that can use native functions and skills.
- **Microsoft Copilot Platform:** Managed services for building enterprise copilots with integration to Microsoft's ecosystem.

2.4.3 Data-Oriented Agent Frameworks

- **LlamaIndex:** Focused on connecting LLMs with external data through sophisticated RAG approaches, with agent capabilities for data retrieval and processing.
- **Haystack:** A framework specialized in building search and question-answering systems with agent capabilities for complex queries.

2.4.4 Specialized Agent Frameworks

- **OpenAI Assistants API:** A managed API service for creating stateful assistants with memory, tool use, and retrieval capabilities.
- **BabyAGI:** A minimalist framework focused on autonomous task management and prioritization.
- **ReWOO:** Implements the "Reasoning Without Output" pattern for more reliable planning and execution.
- **AgentGPT:** A framework for building and deploying autonomous AI agents with a web-based interface.

2.5 Comprehensive Table: Key Components, Patterns, and Frameworks

Pattern	Key Components	Frameworks
Sequential Agent Architecture	Brain: KC1.1 (Foundation LLMs). Control: KC2.1 (Sequential workflows). Reasoning: KC3.1, KC3.3 (Structured planning, CoT). Memory: KC4.1 (In-agent session). Tools: KC5.1 (Basic SDKs). Environment: KC6.1.1 (Limited API access)	BabyAGI; ReWOO; AgentGPT (basic mode)
Hierarchical Agent Architecture	Brain: KC1.1, KC1.2 (Foundation & multimodal). Control: KC2.2 (Hierarchical planning). Reasoning: KC3.1 (Structured planning/execution). Memory: KC4.3, KC4.4 (Cross-session capabilities). Tools: KC5.2 (Managed platforms). Environment: KC6.1.2, KC6.4 (APIs, data sources)	Amazon Bedrock Agents; Microsoft Copilot Platform; LangChain (orchestrator mode)
Collaborative Agent Swarm	Brain: KC1.1, KC1.2 (Multiple models). Control: KC2.3 (Multi-agent collaboration). Reasoning: KC3.1, KC3.4 (Structured planning, ToT). Memory: KC4.2, KC4.4 (Cross-agent memory). Tools: KC5.1 (Flexible libraries). Environment: Various KC6 capabilities	MetaGPT; CrewAI; AutoGPT (multi-agent mode)
Reactive Agent Architecture	Brain: KC1.2 (Multimodal capabilities). Control: KC2.1 (Responsive workflows). Reasoning: KC3.2 (ReAct paradigm). Memory: KC4.3 (Cross-session). Tools: KC5.3 (Managed APIs). Environment: KC6.3 (Web access)	OpenAI Assistants API; AutoGPT; Microsoft Semantic Kernel
Knowledge-Intensive Agent	Brain: KC1.1, KC1.2 (Foundation & specialized). Control: KC2.1 (Knowledge workflows). Reasoning: KC3.3 (Chain of thought). Memory: KC4.3, KC4.6 (Persistent knowledge). Tools: KC5.1 (Data connectors). Environment: KC6.4 (RAG & data sources)	LlamaIndex; Haystack; LangChain (RAG mode)

Framework Flexibility Notes:

- LangChain appears in multiple patterns as it's highly flexible and can implement various architectural approaches
- Microsoft Semantic Kernel provides components that can support multiple patterns depending on implementation

- AutoGPT can operate in either reactive mode or as part of a collaborative system

This table shows how different frameworks leverage various key components to implement the major architectural patterns in Agentic AI systems.

2.6 Case Studies

2.6.1 Multi-agent co-pilot application

This case study describes a typical copilot application designed using a P3: Distributed Architectures for GenAI Agents (Multi-Agent Systems) approach.

It specifically implements KC2.2 Hierarchical Planning to access various enterprise APIs such as calendar, email, Drive, and Slack. The system is composed of the following collaborating agents:

- **Orchestrator Agent:**
  - **Role:** Acts as the central controller within the hierarchical structure. It receives user requests, utilizes its tools to decompose them into sub-tasks (KC3.1 Structured Planning/Execution principles), and delegates these tasks to specialized agents (Calendar, Email, Slack, Drive) via its Agent Routing Mechanism. It then aggregates the responses from these agents using its Response Aggregator tool to present a unified output to the user.
  - **Core Intelligence:** Leverages KC1.1 Large Language Models (LLMs) within its specialized tools.
  - **Tool Integration:** Likely utilizes KC5.1 Flexible Libraries / SDK Features for API integration.
    - Task Decomposition: An LLM-based tool for understanding user requests and breaking them into smaller, actionable sub-tasks suitable for specialized agents.
    - Agent Routing Mechanism: An LLM-based tool for directing sub-tasks to the appropriate specialized agent based on the task's nature.
    - Response Aggregator: An LLM-based tool for collecting, formatting, and synthesizing responses from specialized agents into a coherent output.
  - **Memory Access:** Utilizes KC4.2 Cross-agent session memory. This memory is shared among the Orchestrator and the specialized agents within the context of a single user session. It stores user requests, intermediate reasoning data, responses from specialized agents, and serves as the primary mechanism for communication between agents during that session. Context "sensitivity" (classification or compartmentalization) is used to reduce risk of unauthorized information exposure.
- **Calendar Agent:**
  - **Role:** Responsible for managing the user's calendar (scheduling, availability checks, reminders).

    - **Operational Environment (Agency Type):** Operates under KC6.1.1 Limited API Access, interacting with the user's calendar service (e.g., Google Calendar) via its specific API. This involves predefined API calls where the agent's LLM might generate some parameters, requiring authentication and authorization.
    - **Specialized Tools:** Date and Time Parsing; Meeting Scheduling Logic; Reminder System.
- **Email Agent:**
    - **Role:** Manages the user's email (sending, receiving, organizing).
    - **Operational Environment (Agency Type):** Operates under KC6.1.1 Limited API Access, interacting with the user's email service (e.g., Gmail, Outlook) via its API, requiring authentication/authorization.
    - **Specialized Tools:** Email API Integration; Email Composition and Sending; Email Reading and Parsing; Email Filtering and Organization.
- **Slack Agent:**
    - **Role:** Facilitates communication and collaboration within Slack.
    - **Operational Environment (Agency Type):** Operates under KC6.1.1 Limited API Access, interacting with the Slack API, requiring authentication/authorization.
    - **Specialized Tools:** Slack API Integration; Message Sending and Retrieval; Channel Management; Notification Handling.
- **Drive Agent:**
    - **Role:** Manages files and documents in the user's cloud storage (e.g., Google Drive, OneDrive).
    - **Operational Environment (Agency Type):** Operates under KC6.1.1 Limited API Access, interacting with the cloud storage service's API, requiring authentication/authorization.
    - **Specialized Tools:** Drive API Integration; File Retrieval and Upload; File Searching and Filtering; File Sharing and Permissions.

Applicable Threats:

- **Tool Misuse (T2) via KC6.1.1 Limited API Access:** Even with predefined API calls, agents' KC1.1 LLMs generate parameters. An attacker crafting deceptive prompts could cause the LLM to generate malicious parameters, potentially leading to API vulnerabilities like parameter pollution attacks or unauthorized operations within the API's allowed scope.
- **Intent Breaking & Goal Manipulation (T6) impacting KC1.1 LLM within KC6.1.1:** Attackers can inject prompts attempting to manipulate the agent's underlying KC1.1 LLM. The goal is to steer the agent towards generating parameters that achieve a malicious goal, even within the constraints imposed by the KC6.1.1 Limited API Access.
- **Identity Spoofing & Impersonation (T9) related to KC6.1.1 parameters:** If the parameters generated by the agent's KC1.1 LLM for the KC6.1.1 Limited API Access calls include user identifiers or potentially manipulable tokens, an attacker might try to alter these parameters via prompt injection to impersonate another user or spoof identity within the scope of the API interaction.

2.7 Cross-Cutting Architectural Concerns

The following concerns are not new key components; they are properties that cut across KC1–KC6 and must be addressed at design time. They are introduced here and made concrete in the lifecycle (*Section 3*), capability (*Section 5*), and deployment (*Section 9*) sections.

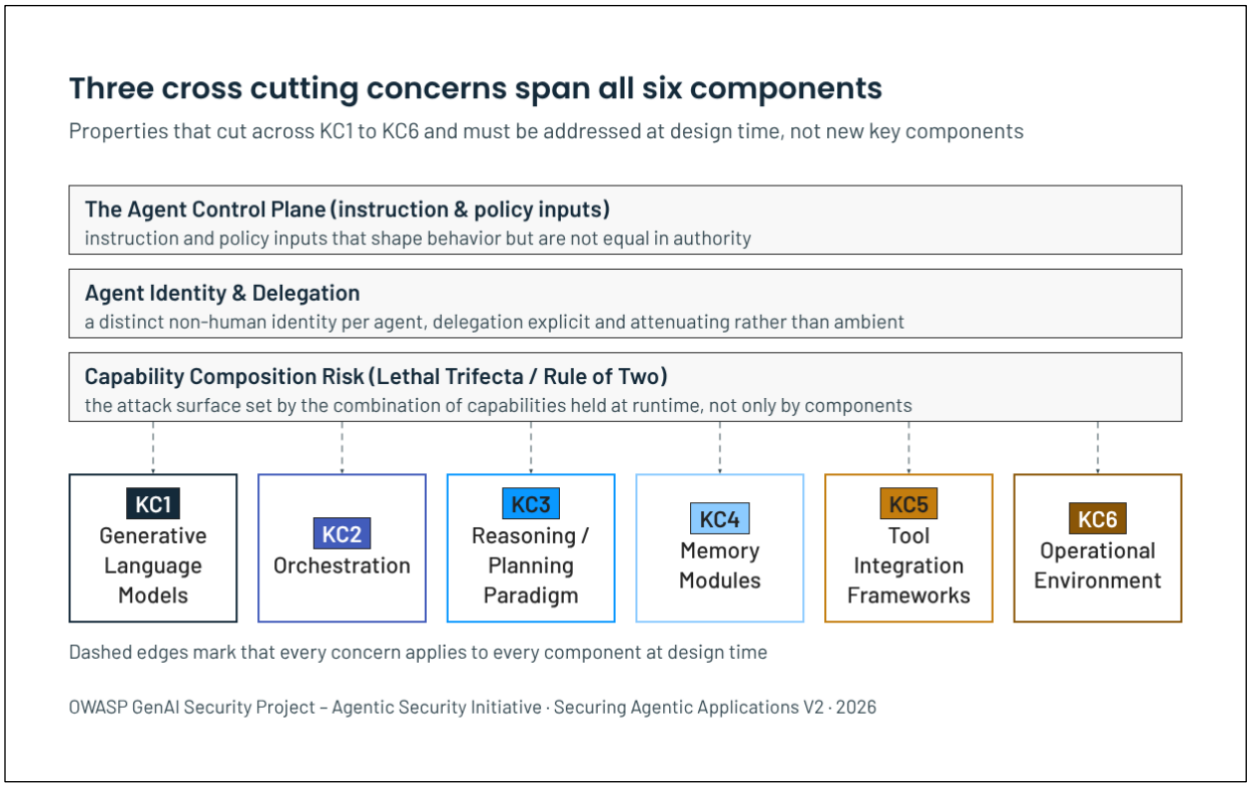


Figure 2.5: The three cross cutting concerns, The Agent Control Plane, Agent Identity & Delegation, and Capability Composition Risk, drawn as a layer over KC1 to KC6. They are properties addressed at design time, not new key components.

2.7.1 The Agent Control Plane (instruction & policy inputs)

System prompts, developer instructions, tool descriptions, retrieved documents, memory summaries, policies, and hook logic are all control-plane inputs that shape behavior and control flow, but they are not equal in authority. Prompt roles, delimiters, and sections are provenance cues rather than security boundaries. Preserve each input's provenance and trust label, route external content (emails, documents, web pages, tool/API responses, and agent messages) through a separately mediated lower-trust ingestion path, and keep deterministic policy and authorization outside the model at action sinks. Verifying active system prompts and tool definitions against approved hashes at session start and checkpoints detects unauthorized control-plane changes (see 3.1.2 and Section 6).

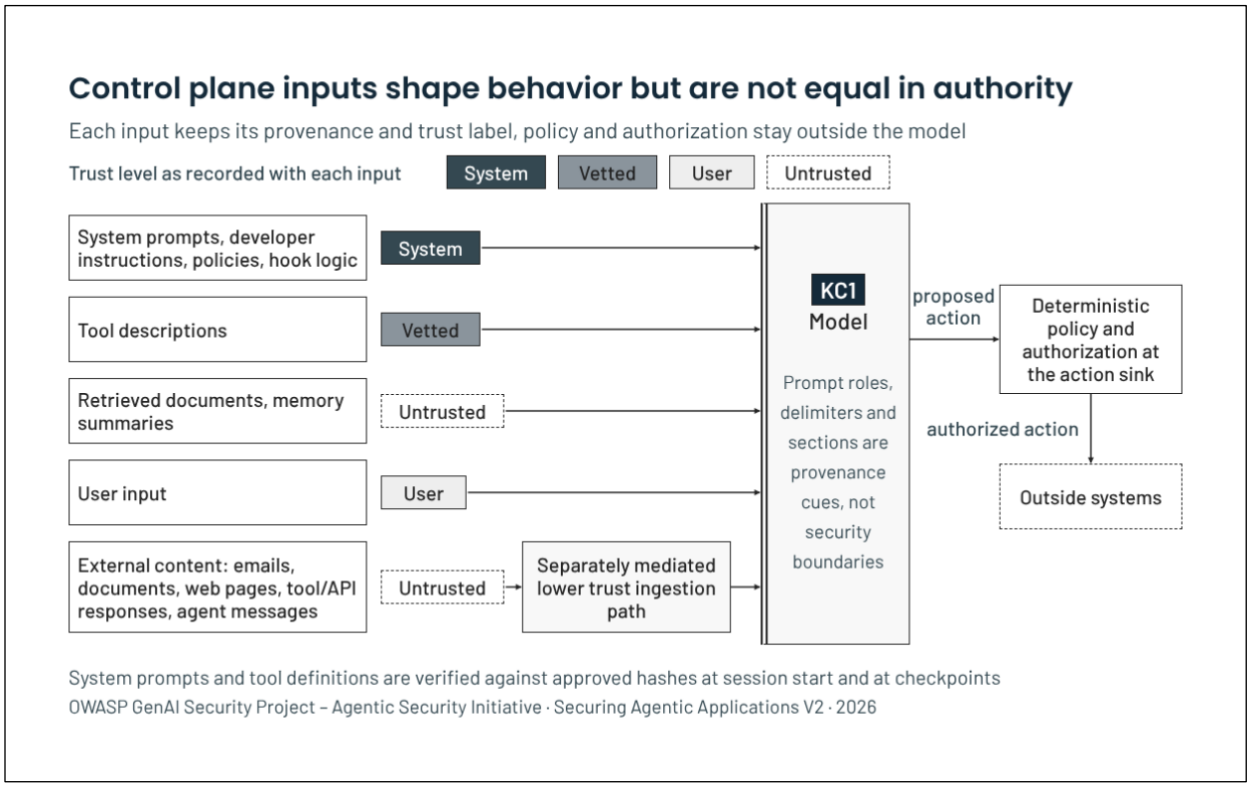


Figure 2.6: Control plane inputs reach the model with their provenance and trust label attached. External content takes a separately mediated lower trust ingestion path, and deterministic policy and authorization sit outside the model at the action sink.

2.7.2 Agent Identity & Delegation

Identity is a cross-cutting control spanning orchestration (KC2), tool integration (KC5), the operational environment (KC6), and deployment. Each agent instance or service should hold a distinct, manageable non-human identity, and authority delegated *to* an agent or *between* agents should be explicit and attenuating rather than ambient. A useful framing is the effective-scope relationship Effective Scope = User Permission ∩ Agent Capability ∩ Bound Intent. Delegation should use On-Behalf-Of (OBO) token exchange (never raw user credentials) with depth limits and monotonically narrowing scope at each hop, and should bind authorization to the declared user intent so that an agent does not become a "confused deputy." These patterns are detailed in *Section 4* and *Section 9*.

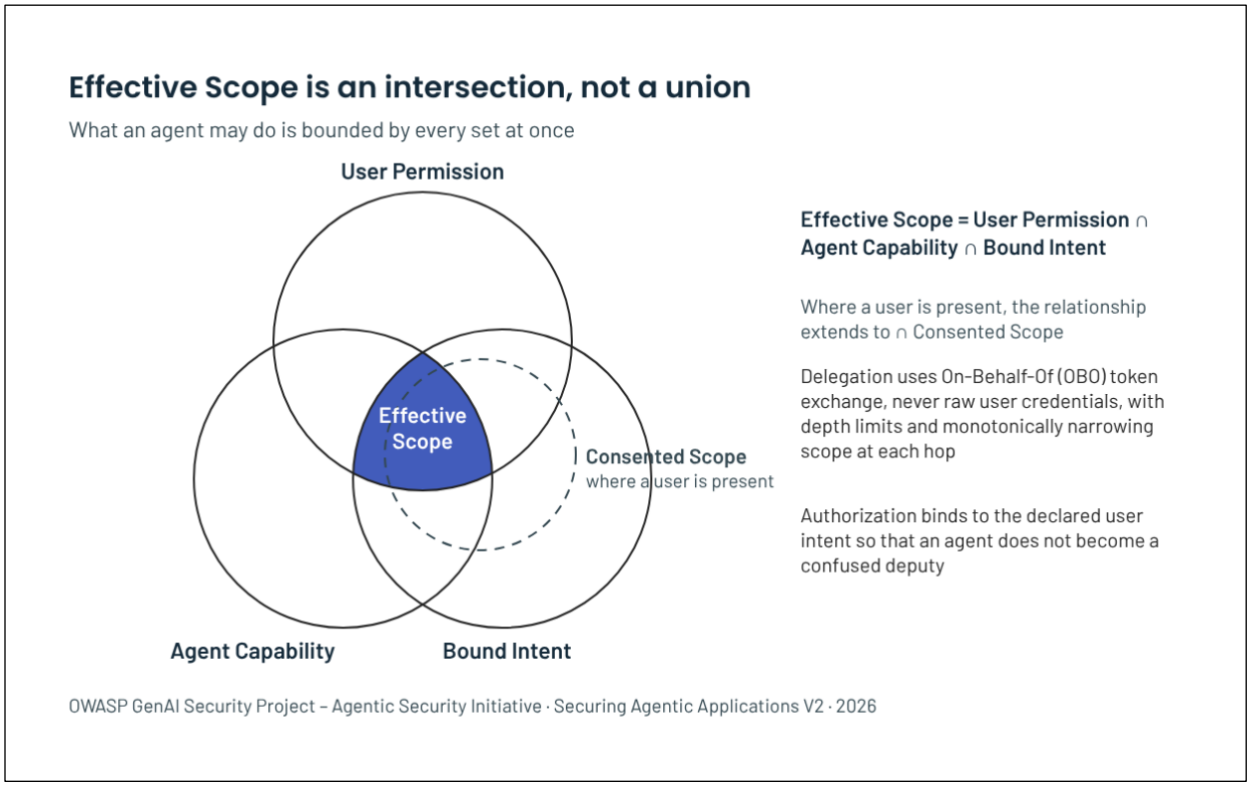


Figure 2.7: Effective Scope = User Permission ∩ Agent Capability ∩ Bound Intent. Where a user is present the relationship extends to ∩ Consented Scope, drawn as the dashed set. Delegation uses On-Behalf-Of token exchange, never raw user credentials, with depth limits and narrowing scope at each hop.

Agent Identity, Delegation, and End-User Consent

Agent identity must distinguish four layers: (1) the logical agent or service role, (2) the runtime workload identity, (3) the ephemeral execution, session, or task identifier, and (4) the delegated end-user principal. Assign a stable, managed non-human identity at an independently deployable workload or service boundary, issue short-lived attested credentials to runtime instances, and create a separately registered principal only

where ownership, isolation, policy, or independent revocation requires it. Telemetry and authorization must map every ephemeral run back to its stable owner, deployment, task, and delegation chain. Effective authority remains Effective Scope = User Permission ∩ Agent Capability ∩ Bound Intent.

Delegation presumes consent. Consent must be explicit, recorded, and revocable. Where a user is present and the agent requires access to user-owned resources, the OAuth 2.0 Authorization Code Grant with PKCE provides an appropriate consent mechanism: the user authenticates at the authorization server, reviews the requested scopes, and the agent receives a short-lived token bound to those consented scopes—never observing the user's credential—extending the relationship to Effective Scope = User Permission ∩ Agent Capability ∩ Bound Intent ∩ Consented Scope.

Where no user is present, the Client Credentials Grant authenticates the agent under its own identity with administrator-bounded scope. In multi-hop chains where a user context already exists upstream, On-Behalf-Of token exchange (RFC 8693) propagates identity downstream with monotonically narrowing scope. The choice among these approaches is driven by data ownership, availability of user interaction, operation timing, and whether permissions should vary per user consent or remain fixed at the agent level.

2.7.3 Capability Composition Risk (Lethal Trifecta / Rule of Two)

The attack surface of an agent is determined not only by individual components but by the combination of capabilities it holds at runtime. Simon Willison's "Lethal Trifecta" identifies the dangerous combination of (1) access to private or sensitive data, (2) exposure to untrusted content, and (3) the ability to communicate externally; an agent holding all three can be manipulated into exfiltrating data. Meta's "Rule of Two" formalizes the mitigation: expose an agent to at most two of {processes untrustworthy input, accesses sensitive systems/data, communicates externally or changes state}. When a design would combine all three, the default is to remove one capability or separate the legs across independently authorized components with hard information-flow and egress boundaries. Any exception requires layered deterministic enforcement, tested information-flow and egress controls, documented residual-risk ownership and acceptance, and evidence that HITL or monitoring is supporting rather than the sole control. This gate is especially important for dynamically acquired capabilities; Section 5 operationalizes it.

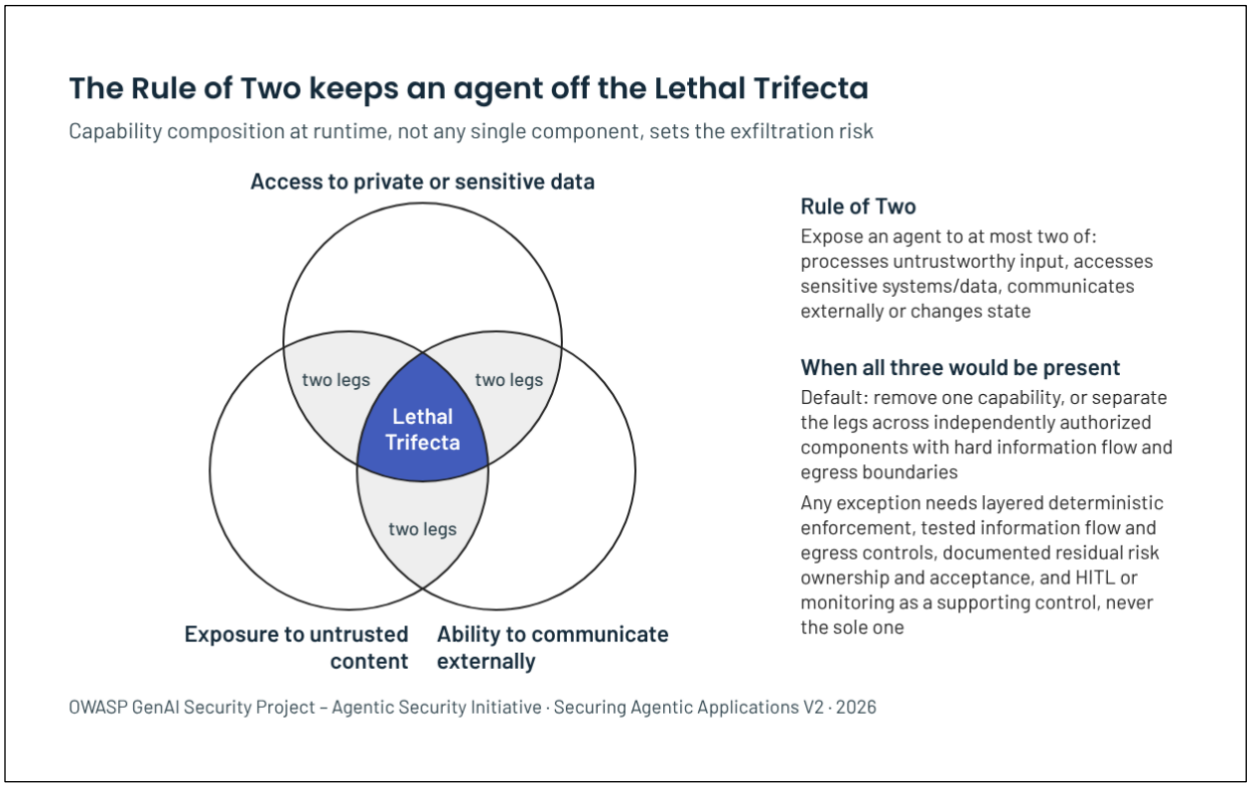


Figure 2.8: The Lethal Trifecta, access to private or sensitive data, exposure to untrusted content, and the ability to communicate externally. The Rule of Two exposes an agent to at most two legs. When a design would combine all three, the default is to remove one capability or separate the legs across independently authorized components.

3. Agentic Developer Guidelines

Agentic AI Developer Security Guidelines: A Lifecycle Approach

Building secure Agentic AI systems requires integrating security considerations throughout the entire development lifecycle, from initial design to ongoing operations. Security isn't an afterthought; it's a foundational principle necessary regardless of the agent's architecture (single, multi-agent, swarm), complexity, or function. Proactive security measures create a resilient framework against evolving threats specific to AI systems, such as prompt injection, insecure output handling, and data poisoning. This guide outlines practical security controls across the typical development phases. Consider referencing frameworks like the OWASP Top 10 for Large Language Model Applications for common vulnerabilities.

3.1 Secure Design & Development Phase

Agentic AI systems introduce fundamentally new security surfaces. Persistent memory raises the risk of context poisoning, data leakage, and unauthorized state retention. Autonomous planning opens the door to misaligned objectives, recursive decision loops, and unpredictable emergent behavior. These risks must be accounted for at design time-not just through traditional threat modeling, but through new considerations like defining "safe failure" states, charting integration boundaries, specifying memory access policies, and ensuring human-in-the-loop safeguards where applicable. These early decisions shape downstream control strategies and lay the foundation for resilient agentic systems.

3.1.1 Threat Modeling for Agentic Systems

For details on threat model for agentic systems please see the latest version of:
https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/

3.1.2 System Prompt Engineering & Hardening

- **Purpose:** Define the agent's core instructions, capabilities, and limitations securely to prevent manipulation and unintended behavior.
- **Practical Approach:**
    - **Clear Boundaries & Safeguards:** Explicitly state DOs and DON'Ts, forbidden topics, operational scope, and ethical guidelines. It is preferred to have a predetermined list of allowable topics, with an implicit deny policy for all other topics. This can be similarly applied to languages. Specifically in a multilingual setting, analyze the linguistic

composition of its expected input, particularly the balance between structured elements (e.g., code, formal queries) and localized natural language.
    - **Robustness against Injection:**
        - Use clear delimiters (e.g., XML tags, triple backticks or custom ones) to separate instructions from user input.
        - Instruct the model to be wary of attempts to override its core instructions.
        - Employ placeholders or structured input formats where user input is inserted, rather than direct concatenation.
        - Use few-shot examples demonstrating desired behavior and rejection of malicious requests.
    - **Role Definition:** Clearly define the agent's persona and purpose to anchor its behavior.
    - **Iterative Refinement:** Test prompts against known injection techniques and refine based on results.
    - **System prompt provenance:** Design-time hardening gives no proof the prompt held during operation. At session init and defined checkpoints, verify the active system prompt's hash against the approved version and alert on any change. Apply the same hash-and-verify to tool definitions and permission scopes (this also detects MCP "rug-pull" mutation of a tool after approval - see *Section 6*).

3.1.3 Secure Coding Practices

- **Purpose:** Implement standard secure coding principles adapted for AI agent development.
- **Practical Approach:**
    - **Input Validation:** Validate all inputs, not just tool arguments (user prompts, API responses, data retrieved from memory). Use allowlists where possible.
    - **Error Handling:** Implement robust error handling that avoids leaking sensitive information in error messages.
    - **Secure Key Management:** Avoid hardcoding secrets. Use environment variables, dependency injection, or dedicated secrets management services.
        - Examples: Dependency injection in AG2, InjectedToolArg in Langchain, managed secrets services (AWS Secrets Manager, Google Secret Manager, HashiCorp Vault).
    - **Least Privilege:** Ensure components run with the minimum permissions necessary.

3.1.4 Content Moderation Integration (Design)

- **Purpose:** Plan for detecting and filtering harmful or policy-violating content in both inputs and outputs early in the design.
- **Practical Approach:**
    - Identify necessary checks (hate speech, PII, toxicity, specific disallowed topics).
    - Evaluate tools based on requirements: Rule-based filters, ML classifiers, external APIs.

        - Examples: Meta's LlamaGuard, cloud services (AWS Comprehend, Azure Content Safety), OpenAI Moderation API.
    - Design how moderation results will trigger actions (block, flag, sanitize, alert).
    - Ensure you test your content moderation against your policies and for correctness.

3.1.5 Human-in-the-Loop (HITL) Design

- **Purpose:** Determine where human oversight is critical before the agent takes high-impact actions.
- **Practical Approach:**
    - **Risk-Based Identification:** Identify actions requiring approval (e.g., sending emails, modifying databases, executing code, spending funds, updating critical memory). Use the EU AI Act's list of high-risk actions to determine additional use cases which require further human oversight.
    - **Clear Workflow:** Design a user-friendly interface for review, approval, or rejection. Log the human decision.
    - **Framework Integration:** Leverage built-in HITL features in agent frameworks.
    - ○ Include strong rate limiting to prevent overwhelming human in a DoS situation.
        - Examples: Langchain Interrupts, Autogen User_Proxy_Agent, CrewAI Human Input.

3.1.6 Memory Security Design

- **Purpose:** Plan how to protect the agent's short-term and long-term memory (e.g., vector databases, caches) from unauthorized access, tampering, and data leakage.
- **Practical Approach:**
    - **Access Control:** Use standard mechanisms (IAM roles, API keys, database permissions) to restrict access based on the principle of least privilege.
    - **Treat Retrieved Memory as Untrusted:** Preserve provenance and trust labels, authorize memory writes, validate structure and content as defense in depth, and reapply deterministic policy before retrieved data may influence a consequential action. Keyword or regex filtering is only a detection signal and must not be treated as sanitization.
    - **Encryption:** Encrypt data both at rest (within databases/storage) and in transit (during retrieval/updates).
    - **Input Validation for Memory:** Validate and sanitize data before storing it in memory to prevent storing malicious content or corrupted data (OWASP Input Validation Cheat Sheet).
    - **PII Handling:** Design mechanisms for PII detection and redaction/anonymization before storage (see Runtime section for tools).
    - **Integrity & provenance (and their limits):** Use HMACs or integrity hashing on memory records, tag every entry with provenance (source, author, timestamp, trust level), and scan content for instruction-like patterns before persisting. Note the limit precisely: integrity checks *detect unauthorized modification* of a stored record; they do **not** prove the content is

semantically safe, and they do **not** by themselves prevent legitimately-written-but-poisoned content (e.g., "sleeper agent" instructions) from influencing later reasoning. A cryptographically intact record can still be malicious. Integrity hashing must therefore be paired with semantic content validation and provenance/trust labeling so the agent can distinguish trusted system inputs from untrusted user/external content before acting on retrieved memory. Treat poison that is planted now but triggers later on semantic similarity (temporal decoupling) as part of the threat model, and monitor for memory writes that originate from processed content rather than explicit user commands.

3.1.7 Input/Output Validation & Sanitization Design

- **Purpose:** Define strategies to ensure the integrity and safety of data flowing into and out of the agent and its tools.
- **Two layers of validation:** It is useful to separate validation into two distinct gates, because they protect different things and run at different points in the loop:
    - **Context admission validation** - applied *before untrusted content enters the model context*. Applies to user input, web pages, RAG results, tool/API outputs, MCP tool descriptions, and agent-to-agent messages. The goal is to keep instruction-like or malicious content from being admitted as if it were trusted instruction (see the control plane discussion in *2.7.1*). Because fragmented multi-layer injection payloads can evade content filters, schema enforcement and runtime isolation should be prioritized over content sanitization alone.
    - `Multimodal context admission - apply the same trust and provenance policy to non-text modalities. OCR and speech-to-text convert untrusted media into another untrusted representation; they do not sanitize it, and a multimodal model may perceive content that extraction misses. Preserve provenance linking media, metadata, and derived text; use extraction and pattern scanning only as defense-in-depth signals; and process untrusted media in a no-tools or low-privilege component before modality-aware policy gates any downstream action.`
    - **Action execution validation** - applied *before anything affects the outside world*. Applies to API calls, code execution, database writes, email/send actions, file writes, browser actions, and OS commands. The goal is to confirm the action is authorized, in scope, and matches the agent's declared intent before it executes.

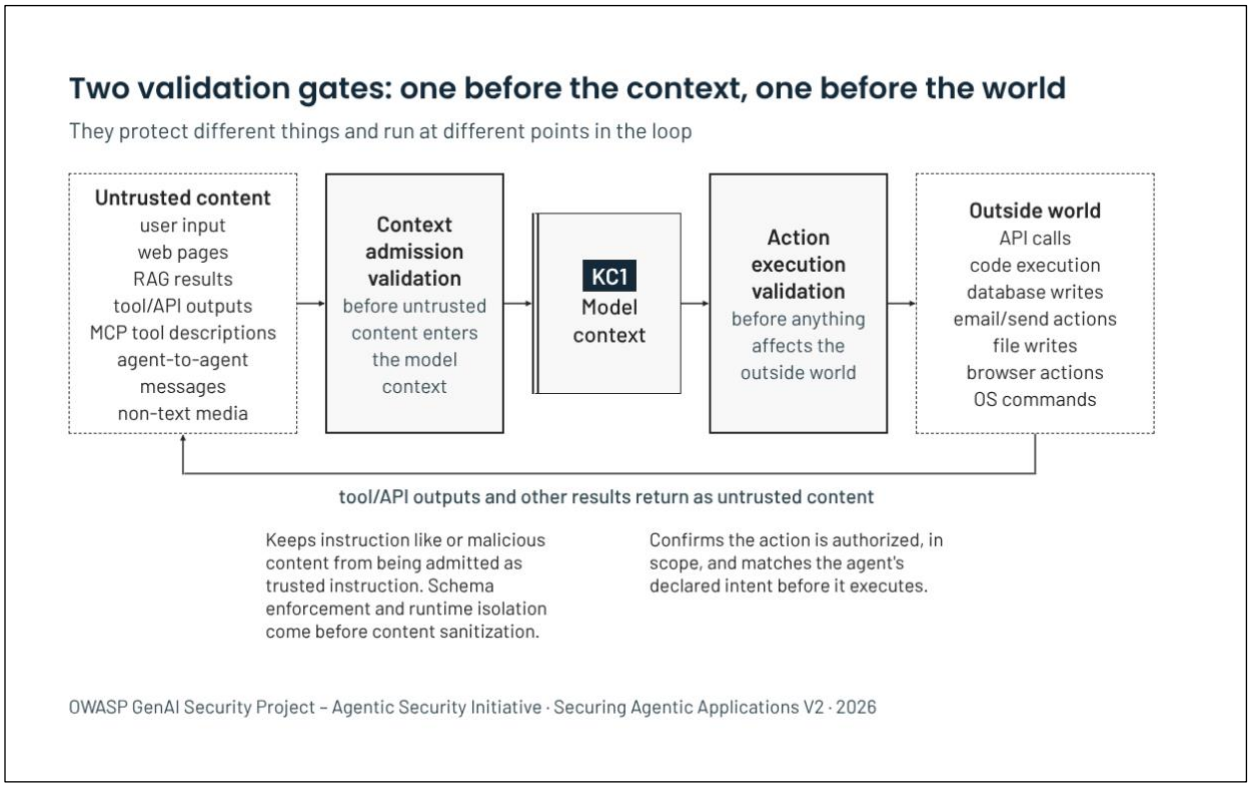


Figure 3.1: The two validation gates of 3.1.7. Context admission validation runs before untrusted content (user input, web pages, RAG results, tool/API outputs, MCP tool descriptions, agent-to-agent messages, non-text media) enters the model context. Action execution validation runs before anything (API calls, code execution, database writes, email/send actions, file writes, browser actions, OS commands) affects the outside world. Results return as untrusted content.

- **Practical Approach:**
  - **Input Sanitization and Escaping:**
    - Escape or strip special tokens such (##, <<, {{}}), etc. before injecting user input into prompts.
    - Avoid prompt templating patterns that concatenate raw user input into instructions.
  - **Schema Enforcement:** Define strict schemas for tool arguments and expected outputs. For example, use structured formats like JSON with string parsing to constrain the output to a predictable schema and possibly reduce the risk of unexpected reinterpretation or hallucinating new instructions.
    - Example: Reject duplicate JSON member names and other ambiguous encodings before schema validation. Parse once into a canonical typed structure, such as {"actions":[{"type":"tell_joke"}]}, then separately authorize every normalized

operation against the authenticated actor, intent, target, resource, and current policy at the execution sink; schema conformance is not authorization.
        - Examples: Pydantic, JSON Schema, OpenAI Structured Outputs.
    - **Allow/Deny Lists:** Use allow lists for known-good values and deny lists for known-bad patterns in tool arguments or outputs where applicable.
    - **Output Sanitization Strategy:** Plan how to neutralize potentially malicious content (e.g., scripts, HTML) in LLM outputs before rendering or passing to other systems. Consider context (e.g., rendering in a browser vs. saving to a database), dropping outputs with suspicious content, blocking payloads that do not match expected structure, disallowing further propagation of injected payloads (i.e.: ensuring the malicious input does not spread or get reused by the Agent or its environment). Use WAFs where possible.
        - Examples: JSON schema; custom filters that enforce allowed fields/types/values (depending on the use case and without being too restrictive).
    - **Secure Rendering:** Plan to use safe methods for displaying agent output in UIs (e.g., element.textContent instead of element.innerHTML in JavaScript).
    - **Facilitate Human Review:** In regulated scenarios or where applicable, facilitate Human-in-the-Loop (HITL) via JSON logs of outputs which are easier to audit and can support automated review, tracking of field-specific anomalies, and correlation of risk patterns.
    - **Multilingual adaptation:** Minimize low-resource language usage for the input, keep low proportion of it, by translation or adding additional english equivalent content in the system prompt.

3.1.8 Authorization and Authentication

- **Purpose:** Authenticate and authorize agentic systems to perform actions and read data on behalf of users or systems.
- **Practical Approach:**
    - **Identify Permission Boundaries:** Examine agent & tools for expected actions and data reads. Ensure agents & tools are coded to respect their permissions.
    - **Use Identity Providers and Authorization Servers:** Use existing identity and authorization frameworks to support new agent development. Determine if an agent is acting as a business system or on behalf of a user and create the correct identity type for the use case.
    - Use Decentralized Identifiers Only Where Interoperability Requires Them: A DID is an optional identifier and verification-method discovery mechanism; resolution alone neither authenticates a caller nor authorizes an action. Require fresh proof of control using an allowed verification method, validate method-specific trust, issuer, audience, freshness, key rotation and recovery, and revocation or status, then apply application authorization to the requested action and resource.
    - **Credential Providers & Token Vaults:** When agents need to access services with separate trust domains (e.g., GitHub, external calendars), avoid persistent credential storage.

Instead, use a Token Vault pattern: agents retrieve short-lived, scoped access tokens from a secure vault (e.g., HashiCorp Vault, AWS Secrets Manager) based on the agent's verified identity and specific task requirements. This prevents credential sprawl and reduces the risk of pivoting across trust domains.

3.2 Secure Build & Deployment Phase

As organizations move toward productionizing agentic applications, a robust and adaptive development lifecycle becomes essential. Development environments should sandbox agent chains to isolate failures, constrain tool usage through strict policy checks, and guard against memory corruption or drift. Deployment pipelines must go beyond packaging and release-they need to include snapshotting agent capabilities, logging audit trails, and embedding signed constraints on tool/API usage and memory schema. Release protocols should support rollback, serialization integrity, and runtime enforcement of behavioral boundaries to ensure safe deployment into real-world systems.

3.2.1 Static Analysis & Code Scanning (SAST)

- **Purpose:** Automatically detect potential security vulnerabilities in the agent's source code before runtime.
- **Practical Approach:**
    - Integrate SAST tools into the CI/CD pipeline to scan code on commit/merge.
    - Focus on common web vulnerabilities (if applicable), insecure use of APIs, hardcoded secrets, and unsafe code patterns.
    - Examples: Bandit (Python), Semgrep (multi-language), ESLint + security plugins (JavaScript), Meta's PurpleLlama CodeShield (focused on secure code generation).

3.2.2 Dependency Vulnerability Scanning (SCA)

- **Purpose:** Identify known vulnerabilities in third-party libraries and dependencies used by the agent.
- **Practical Approach:**
    - Integrate SCA tools into the CI/CD pipeline or use package manager features.
    - Regularly scan dependencies and update libraries with known vulnerabilities.
    - Examples: Snyk, GitLab Dependency Scanning, pip-audit, GitHub Dependabot.

3.2.3 Environment Hardening & Sandboxing

- **Purpose:** Isolate the agent's execution environment to limit the potential impact of a compromise.
- **Practical Approach:**

- **Sandboxing:** Execute agent code, especially code generated by the LLM or tools involving execution (like Python interpreters), in isolated environments. Choose based on isolation needs vs. performance overhead.
  - OS-level Containers: Docker, Podman - Good balance for many applications.
  - VM/MicroVM: Firecracker, QEMU/KVM - Stronger isolation with separate kernels.
  - WebAssembly (Wasm): Provides memory safety, suitable for client-side or constrained server-side execution. (NVIDIA Wasm Blog)
  - Sandboxed Interpreters: e.g., Pyodide.
  - Cloud Sandboxing Services: Provide managed secure execution environments. (E2B Example)
  - Research Example: MITRE's OCCULT framework uses a simulated CyberLayer for testing LLM capabilities safely (arXiv:2502.15797).
- **Filesystem/Network Restrictions:** Configure strict permissions within the sandbox (read-only filesystem where possible, limited network egress/ingress).
- **Principle of Least Privilege:** Run agent processes with minimal OS-level privileges.
- **Prefer deterministic boundaries over probabilistic safety:** Prompt engineering is a best-effort safety layer, not a primary security control; relying on the model to self-restrict is insufficient. Enforce isolation deterministically and externally to the model: kernel isolation (gVisor, Firecracker, QEMU/KVM microVMs) rather than plain containers where the threat model warrants it; syscall restriction via seccomp / AppArmor / SELinux, with eBPF for fine-grained syscall monitoring without kernel-module overhead; network egress filtering with host allowlists; and path isolation via mount namespaces, read-only volumes, and ephemeral tmpfs. See *Section 9* for the runtime deployment of these controls.
- Execution Integrity: Passing identity, policy, and validation checks does not guarantee that a previously approved action remains authorized at execution. Treat authorized_request == executed_request as a semantic invariant, not byte-for-byte equality: bind a signed execution artifact to the authenticated actor and delegated subject, canonical action or tool, verified target and tenant or object, approved parameters, resource or state version, policy version, audience, expiry, and a unique transaction identifier or nonce. Immediately before each side effect, the execution sink must authenticate the caller, reauthorize against current state and policy, and reject expired, replayed, revoked, stale, or materially changed artifacts.

3.2.4 Secure Configuration Management

- **Purpose:** Securely manage configurations, especially secrets and API keys, during deployment.
- **Practical Approach:**
  - **Secrets Management:** Use dedicated systems (AWS Secrets Manager, Google Secret Manager, Vault) instead of config files or environment variables for sensitive data. Implement credential rotation (OWASP Secrets Management Cheat Sheet).

- **API Access Control:** Use fine-grained access controls for external APIs (e.g., OAuth2 scopes (https://oauth.net/2/scope/), restricted API keys) granting only necessary permissions.
- **Infrastructure as Code (IaC) Security:** Scan IaC templates (Terraform, CloudFormation) for security misconfigurations before deployment.

3.2.5 Pre-deployment Testing (Fuzzing, Pen Testing)

- **Purpose:** Actively probe the agent and its interfaces for vulnerabilities before release.
- **Practical Approach:**
  - **Prompt Interface Fuzzing:** Use automated tools to send malformed, unexpected, or adversarial inputs to the agent's prompt interface to uncover injection vulnerabilities or unexpected behavior.
    - Examples: PromptFoo, PyRIT, Garak.
  - **API Fuzzing:** Test any custom APIs exposed by the agent system.
  - **Targeted Penetration Testing:** Conduct manual or automated penetration tests focusing on the specific risks identified during threat modeling.

3.3 Secure Operations & Runtime Phase

Live agentic systems require continuous behavioral monitoring that extends far beyond performance metrics. Runtime observability must capture decisions, plan evolution, memory access patterns, and agent-tool interactions. Behavioral drift-such as deviation from expected plans or misuse of tools-must trigger alerts or invoke safeguards. Agents should log timestamped reasoning traces for post-incident analysis, and operators must retain real-time override capabilities. Monitoring should create trust through full visibility into agent behavior, particularly around memory usage, planning shifts, and inter-agent communication.

3.3.1 Continuous Monitoring & Anomaly Detection

- **Purpose:** Detect malicious activity, policy violations, and deviations from expected behavior in real-time.
- **Practical Approach:**
  - **LLM Input/Output Scanning:** Monitor prompts and responses for:
    - Jailbreak Techniques: Use classifiers, heuristics (pattern matching for repetition, instruction overrides, role-play attacks), or external APIs to detect known attacks. (LlamaGuard Classifier, NeMo Guardrails Heuristics, OpenAI Moderation API).
    - Policy Violations: Check against content policies (see Moderation).
    - PII Detection: Flag potential PII in inputs/outputs.

- **Tool Invocation Monitoring:** Log all tool calls, parameters, and results. Monitor unusual frequency, suspicious parameter values, or calls to sensitive tools. Implement rate limiting per session/user (LangChain max_iterations) and timeouts (LangChain max_execution_time).
- **Plan & Execution Monitoring:**
  - Pre-execution Checks: Validate generated plans for coherence, feasibility, safety, and policy alignment before execution. Use rule-based checks or even another LLM ("LLM as a Judge"). Consider dry runs for high-risk actions in a sandbox.
  - Runtime Monitoring: Observe code execution within sandboxes for forbidden actions (e.g., network calls, file system writes).
- **Memory Anomaly Detection:** Monitor memory update patterns for unusual frequency, size, or content potentially indicating poisoning or tampering.
- **Behavioral Analysis (Multi-Agent):** Monitor communication patterns and actions between agents for collusion, manipulation, or unexpected emergent behavior.
- **SIEM Integration:** Feed logs and alerts into a Security Information and Event Management (SIEM) system for centralized analysis and correlation.

3.3.2 Runtime Guardrails & Automated Moderation

- ·**Purpose:** Enforce policies and constraints dynamically during agent operation.
- **Practical Approach:**
  - **Input/Output Guardrails:** Implement real-time checks to block or sanitize inputs/outputs based on predefined rules (prompt injection and jailbreak detection, keyword blocking, pattern matching, policy enforcement).
    - Examples: Nemo Guardrails, OpenAI Agents SDK Custom Guardrails, several commercial vendors.
  - **Locate guardrails:** The proximity of the guardrail relative to the application code and LLM has an impact on threat detection effectiveness depending on the type of threat. For example, a network-based guardrail (e.g. API or AI gateway) may miss multi-turn prompt injection attacks because the network-based guardrail may lack the session history and context which are necessary to detect such an attack. Deploying guardrails within the application and/or agent code space offers the greatest opportunity for full visibility and context, and therefore the maximum potential detection effectiveness.
    - **API Gateway (Input Layer):** Deterministic controls on inputs and access. Ensure only valid, authorized and safe query comes in.
    - **LLM (Model):** Model Alignment and behaviour rules. It goes into directions like System prompt & policies and Fine-Tuning/RLHF.
    - **Agent (Reasoning & Tools):** Orchestration and tools use guardrails. It goes into the direction of Tool permissions, Step-wise validation, Memory Controls etc.

        - **Output (Post Process):** Final output filtering and correction. Output should be safe for the user, correct and compliant, policy, and objective conform.
        - **Workflow consideration:** Given the multi-agent workflow, some agents in the workflow will influence the output more than others, therefore the guardrails placement should take into consideration the agent level of influence. For example in the sequential architecture the last agent will influence the output of the workflow the most.
    - **Content Filtering (Inter-Agent/Tool):** Sanitize data exchanged between agents or between agents and tools/APIs to prevent propagation of harmful or covert content or commands. Apply filtering at the orchestrator level or message handoff points, for example redacting non-printable characters.
    - **Memory TTL (time to live) or Expiration:** Expire old or stale memory content automatically and after a reasonable amount of time so that sensitive data is not persistent and unnecessarily exposed.
    - **Memory Sanitization & PII Redaction:**
        - Use safe memory wrappers: encode past actions in markup/JSON format so that models treat them as data.
        - Implement read-only buffers: split memory into read/write areas. For example, system instructions would be read-only.
        - Sanitize and validate data before storing it in memory (short-term or long-term). Use validation libraries and standard sanitization techniques (escaping, allowlisting), and also clean memory buffers before replying to the user (i.e.: profanity, filter, etc.).
        - Implement robust PII detection and removal/masking before memory storage.
            - Examples: Regex (for structured PII), NER models (spaCy), dedicated services (Microsoft Presidio SDK).
        - Consider memory management tools allowing selective deletion (Mem0, Zep).
    - **Output Sanitization for UI:** Use libraries to sanitize output specifically before rendering it in a web UI to prevent XSS. Configure strictly and test against bypasses (OWASP XSS Cheat Sheet).
        - Examples: DOMPurify, sanitize-html.
    - **Limit data subjects in context:** Purge memory if context changes data subjects, to avoid information leakage, contamination and bias.
    - **Context Window:** Implement context-window bound with access. For example, limit how far back a model can recall or query from past conversations.
    - **Content Security Policy (CSP):** Implement CSP HTTP headers as a defense-in-depth measure in the browser to restrict script sources, styles, etc. (OWASP CSP Cheat Sheet). CSP is used as a second layer of defense to prevent vulnerable web applications from executing unsafe/injected code or reaching out to external malicious sites. AI agents

interacting directly with websites may introduce new vulnerability types or unknowingly execute/interact with malicious code, therefore it is wise to enforce a CSP to limit blast radius.

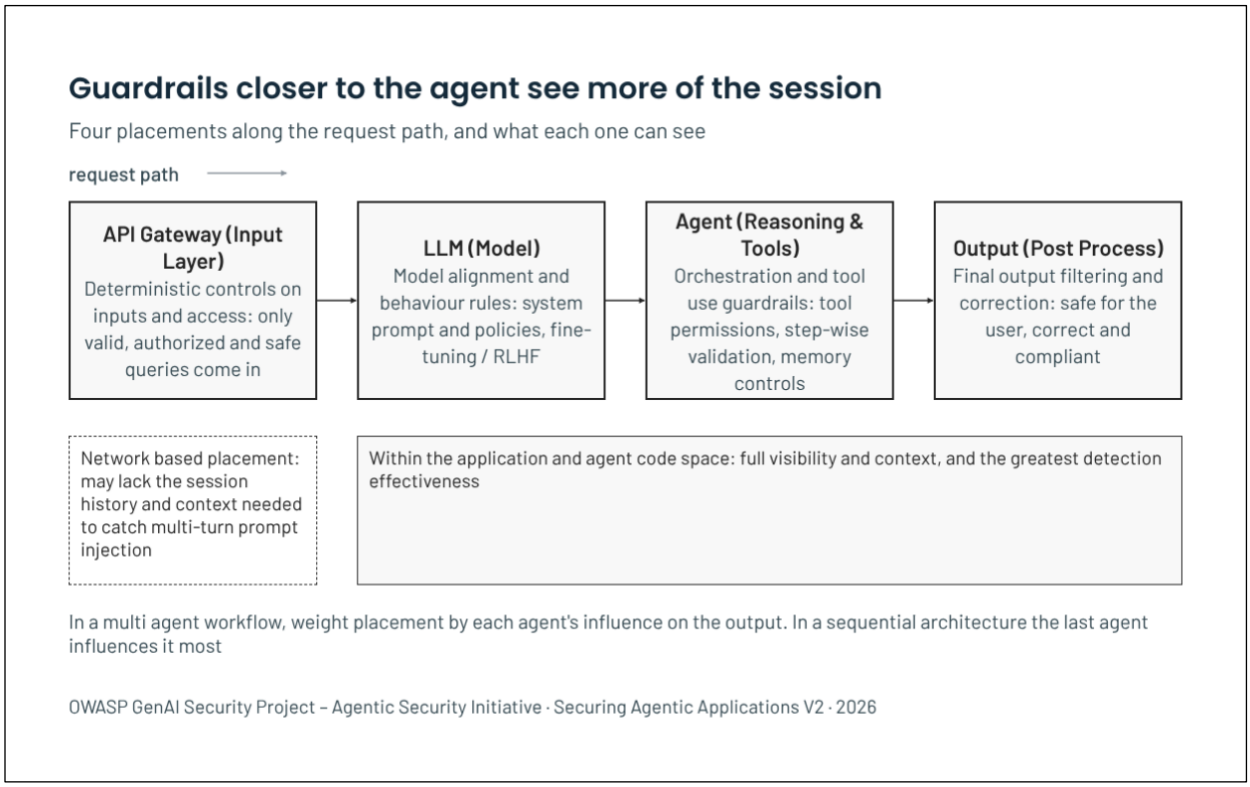


Figure 3.2: The four guardrail placements along the request path, API Gateway (Input Layer), LLM (Model), Agent (Reasoning & Tools) and Output (Post Process), with what each one covers. A network based placement may lack the session history and context to catch multi-turn prompt injection; placements within the application and agent code space see the most.

3.3.3 Logging, Auditing & Traceability

- **Purpose:** Maintain detailed records for debugging, security analysis, compliance, and understanding agent behavior.
- **Practical Approach:**
  - **Session Scoping:** Tag memory buffers by session ID or user identity (JWT, OAuth, etc.) for logging, auditing & traceability.
  - **Comprehensive Logging:** Log agent reasoning steps, generated plans, validation results, tool calls (with parameters/results), HITL interactions (including decisions), errors, and state changes.
  - **Structured Logging:** Use formats like JSON for easier automated parsing and analysis.

- **Traceability:** Implement trace IDs that propagate through agent steps and tool calls to reconstruct the flow of execution.
- **Centralized Logging & Monitoring Platforms:** Utilize platforms designed for ML/LLM observability.
  - Examples: MLFlow, Langfuse, LangSmith, Cloud provider tools (Azure Agent Monitoring, AWS Bedrock Trace Events).

3.3.4 Vulnerability Scanning (Runtime)

- **Purpose:** Continuously scan the running application and environment for new vulnerabilities.
- **Practical Approach:**
  - Perform periodic infrastructure vulnerability scans.
  - Integrate findings into a patch management process.
  - Utilize DAST scanning against lower environments for APIs and other common interfaces like OWASP ZAP.
  - Use port and CVE scanners for environment endpoints.

3.3.5 Incident Response Planning

- **Purpose:** Have a predefined plan to handle security incidents involving the AI agent.
- **Practical Approach:**
  - Define what constitutes an incident (e.g., successful prompt injection causing harm, data breach via agent, agent performing unauthorized actions).
  - Outline steps for containment (e.g., disabling the agent, restricting tool access), analysis (using logs and monitoring data), remediation, and reporting.
  - Assign roles and responsibilities for incident response.
  - Regularly test the plan using table tops and "fire" drills.

By embedding these security practices across the design, development, deployment, and operational phases, developers can build more robust, trustworthy, and resilient Agentic AI systems. Security is an ongoing process requiring continuous vigilance and adaptation to new threats.

3.3.6 AI Bot Mitigation and controls

Why identity matters: Bots have long gone beyond simple crawling - we see ticket-buying bots, price-sniping bots, or automated trading bots. AI agents are the next leap: they reason in real time, adapt to context, and can chain multiple high-impact actions: submitting checkout flows, moving funds, provisioning cloud resources, or even spawning other agents. If an attacker spoofs their identity, they can siphon data, commit fraud, or inject malicious commands into your pipeline.

A tamper-proof Agent identity layer empowers:

- Site owners decide whether to allow, throttle, or block each request;
- Other agents and APIs verify who is calling before sharing data;
- The builder keeps their runtime safe from hostile automation and avoids landing on bot-mitigation blocklists.

In short, verifiable Agent identity is the admission ticket to the Agentic Web: build it in now, and your agent is welcomed as a trusted collaborator; neglect it, and you'll be treated as just another threat to be blocked.

Adopt a multi-factor identity bundle - no single signal is sufficient on its own.

Layer	What to do	Why is it spoofing resistance?	Readiness & Strength
Stable network & UA metadata	Serve from a documented IP range and a verbose User-Agent string, including optional headers: X-AI-Agent, X-AI-Agent-ID/Signature.	Helps mitigation systems correlate requests with the cryptographic identity.	Good, but not enough. Proven way for good bots for a decade, but not bulletproof (shared IPs, spoofable UA).
Cryptographic request signing	Sign every HTTP request with [RFC 9421] Message Signatures; expose the verification key at /.well-known/agent-public-key.json. Web-bot-auth is an emerging IETF standard protocol that supports it.	Keys cannot be forged without compromise.	Great for preventing spoofing or MITM attacks, but still in early stages.
Agent Name Service (ANS)	Publish an ANSName that encodes protocol, capability, provider & version; register the name + public key in an open Agent Registry.	Standardised, PKI-backed lookup prevents collisions and impersonation.	Spec v1.0 published; pilot registries and tooling now emerging. Will be ideal for a complete handshake.

Note: Implementing the full identity-signal bundle establishes a unified, trust-rich handshake between your agent and every online asset it touches - streamlining integrations, clarifying intent, and giving both sides precise control over how the interaction unfolds.

Industry Observations:

- The emerging HTTP Message Signatures + mTLS pattern is now used for production-grade bot verification;
- LOKA's Universal Agent Identity Layer and the open-source Agent Network Protocol (ANP) both point toward a PKI-backed model with capability attestations.

Playing nicely with bot-mitigation solutions: Bot-mitigation platforms differentiate good automation from malicious traffic. To stay off the blocklist:

1. **Identify transparently** - ship the identity bundle above on every request.
    - Declarative User-Agent, own your IP Address (if possible), and adopt a cryptographic mechanism. ASN should be adopted once available.
2. **Declare intent, respect policy** - The more an agent states its intent, the easier it is for mitigation services to allow it to work. Advertise purpose via X-Agent-Intent (crawl, pay, assist, ...).
3. **Anticipate the next trust tier** - Authenticated agents will present a stable agent ID, tag each session uniquely, and build a verifiable reputation that downstream systems can query.

Authentication and authorization guidance lives in *3.1.8*, *4.1.1*, *4.2.2* (OBO delegation & intent binding), and the agent-identity cross-cut in *2.7.2*.

3.3.7 Security Hooks (Enforcement Points)

Many of the controls in this guide - guardrails, logging, HITL, memory protection, tool execution control, circuit breakers - only become enforceable if the agent harness exposes reliable interception points in its execution loop. A **hook** is a point in the agent's code-level execution loop where a builder can wire logic that observes an action and returns a disposition before the action takes effect. Hooks are an implementation *mechanism*, not a policy engine: the hook is the interception point, while the guardrail/policy is the separate logic that runs there. A guardrail that cannot observe a tool call *before* it executes is documentation, not a defense.

Dispositions

When a hook fires, the enforcement logic returns a **disposition** - the decision about what happens to the intercepted action. Support five:

- **Allow** - the action proceeds as-is.
- **Deny** - the action is blocked; the agent receives an error or a policy-defined fallback.
- **Modify** - the action proceeds with altered content (e.g., PII redacted from tool arguments). This lets a builder redact one field rather than block an entire tool call.
- **Escalate (HITL)** - the action pauses for human approval before proceeding.

- **Defer** - the action is suspended when the enforcement layer cannot make a confident decision, pending additional context or validation. (E.g., an agent rotating credentials outside a maintenance window might be legitimate - suspending until someone confirms is safer than guessing.)

These five are what a hook returns when it *can* reach a decision. When it cannot - an error, a timeout, or an unreachable enforcement service - the action must **fail closed** (denied or escalated, never allowed through by default). A hook that fails open can be bypassed by anyone able to make it fail.

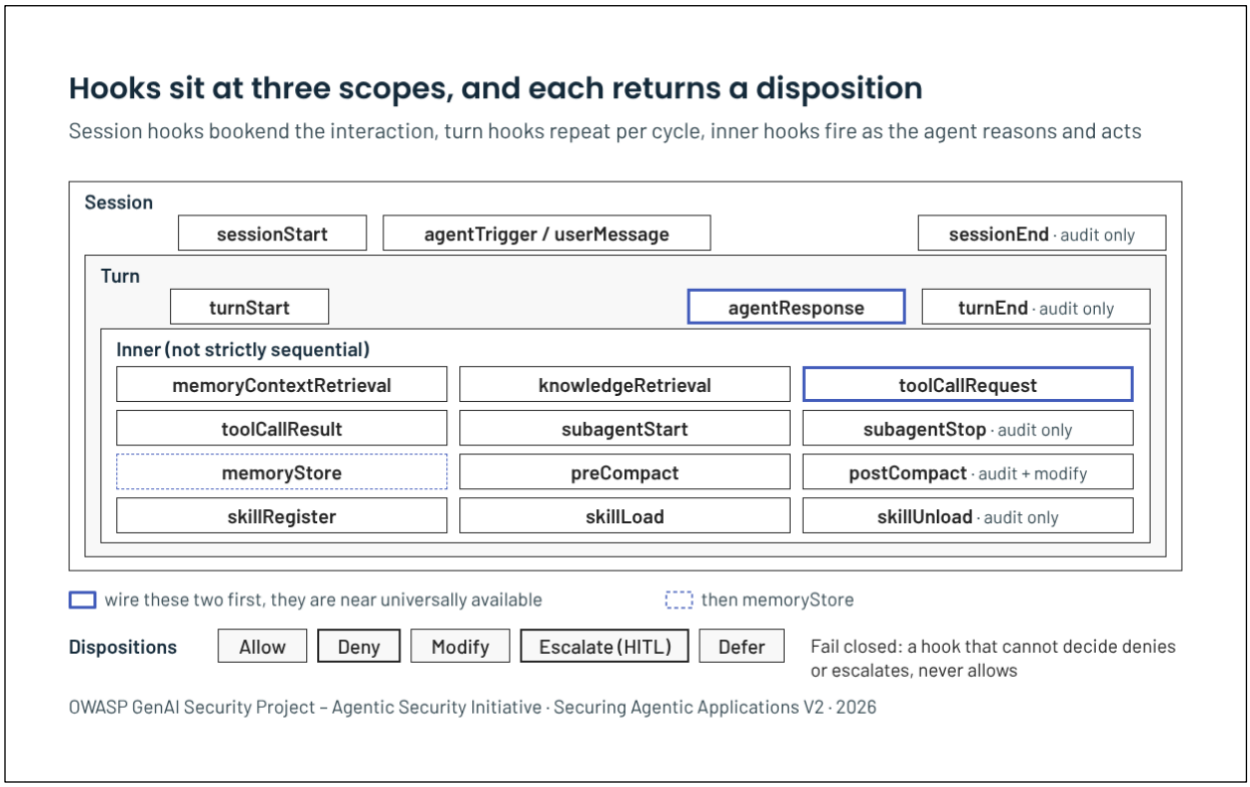


Figure 3.3: The 18 hook points of 3.3.7 by scope. Session hooks bookend the interaction, turn hooks repeat per user to agent cycle, and inner hooks fire within a turn in no fixed order. toolCallRequest and agentResponse are the two to wire first, memoryStore next. Every hook returns Allow, Deny, Modify, Escalate (HITL) or Defer, and fails closed when it cannot decide.

Hook Point Reference

The agent lifecycle has session-level hooks that bookend the interaction, turn-level hooks that repeat per user-to-agent cycle, and inner hooks that fire within a turn as the agent reasons and acts. The inner loop is not strictly sequential - tool calls, sub-agent delegation, memory writes, and compaction interleave depending on the agent's reasoning.

Hook	Scope	Controls it enables	Dispositions
sessionStart	Session	Component inventory check, session policy binding, intent declaration	Allow / Deny
agentTrigger / userMessage	Session	Prompt-injection detection, PII redaction, input policy; trigger-source validation for autonomous agents	Allow / Deny / Modify
turnStart	Turn	Per-turn state init, cumulative-taint reset, turn-scoped rate limits	Allow / Deny
memoryContextRetrieval	Inner	Cross-session taint check, scope enforcement, integrity verification	Allow / Deny / Modify
knowledgeRetrieval	Inner	Retrieved-content filtering, source-provenance tagging, data classification	Allow / Deny / Modify
toolCallRequest	Inner	Permission check, argument validation, MCP hash-pin verification (6.4.2), description-poisoning scan (6.4.3), HITL approval, intent-based access control, context-aware action classification, Rule-of-Two enforcement	Allow / Deny / Modify / Escalate / Defer
toolCallResult	Inner	Output sanitization, exfiltration detection, result-provenance tagging	Allow / Deny / Modify
subagentStart	Inner	Delegation authorization, privilege scoping, intent-inheritance policy	Allow / Deny
subagentStop	Inner	Result-provenance binding, trust-level verification	Audit only

memoryStore	Inner	Write authorization, content filtering, scope restriction, TTL	Allow / Deny / Modify
agentResponse	Turn	Output filtering, data-leakage prevention, hallucination check, compliance	Allow / Deny / Modify
preCompact	Inner	Provenance snapshot before summarization, compaction policy	Allow / Deny
postCompact	Inner	Post-compaction lineage binding, summary-trust verification	Audit + Modify
turnEnd	Turn	Turn summary, step-count audit, cumulative-taint capture	Audit only
sessionEnd	Session	Audit-chain finalization, session summary	Audit only
skillRegister	Inner	Whole-definition inspection before a skill is loadable, signature/provenance check, dependency review, capability-manifest least-privilege, integrity-digest pinning	Allow / Deny
skillLoad	Inner	Bind activation to an approved registration (digest match), inter-skill cascade containment, load-depth limits, capability re-check	Allow / Deny
skillUnload	Inner	Active-inventory update, load/unload churn anomaly detection	Audit only

Framework coverage is uneven. Input interception, tool-call request/result, and agent response are close to universal across major SDKs. Memory read/write, knowledge retrieval, sub-agent lifecycle, and context-compaction hooks remain sparse and inconsistent; no framework yet exposes a hook on the skill lifecycle (a skill load is caught today only because it runs as a tool call). Assess which events your framework can intercept, identify the gaps, and prioritize closing them.

Where to start

If you wire only two hooks, wire toolCallRequest and agentResponse.

- toolCallRequest closes the gap between "agent decides to call a tool" and "tool executes" - the gap exploited by data-exfiltration injection attacks, and the place to inspect model-generated arguments for directory-traversal sequences and unvalidated symlinks before they reach a high-privilege write.
- agentResponse covers the next-highest-damage failure mode: unfiltered sensitive data reaching the user.
- memoryStore is next, because cross-session poisoning depends on writing tainted content to persistent memory without review. Where a framework does not expose it, reach the same protection by screening tainted content where it enters (tool results, retrieved content, user input) and at any memory write that runs through a tool call.

These first two are the ones that are near-universally available. Specify per-call vs. per-batch semantics for toolCallRequest, and streaming-disposition semantics for agentResponse.

Make hooks a hard boundary

A prompt-layer guardrail ("do not call this tool without approval") is a soft boundary the model can ignore under adversarial pressure. A code-layer hook *can* be a hard boundary, but placement alone does not make it one. A hook is a hard boundary only when **two conditions** hold:

1. **Coverage** - it sits on *every* path to a consequential action. Coverage is the condition that fails quietly: a path can reach a consequential action without passing through any hook. During design and testing, enumerate the sensitive leverage points and confirm each sits behind a hook. Paths that most often escape coverage: direct LLM or tool API calls made outside the harness; sub-agents launched in separate processes; MCP servers that call downstream services; tool-initiated callbacks; browser or code-execution environments; background jobs; external plugins that write to memory; and streaming output released before the response is inspected. For paths that run *outside* the harness entirely (a sub-agent in its own process, an MCP server calling downstream), no in-harness hook can reach them - fall back to controls that don't rely on the model's cooperation: credential scoping, network-egress controls, sandboxing, and server-side authorization (see *Section 9*).
2. **Resistance to manipulation** - the decision at the hook cannot be subverted through the channel it polices. A deterministic rule cannot be argued with; a trained classifier scores the input rather than following it; an LLM-as-judge is only as strong as its criteria are tight. Where the decision is manipulable, the action behind it needs a firmer control as well.

Make the hook surface **explicit**: a harness that enumerates the hook points it implements (and, by omission, the ones it does not) turns a silent gap into a declared one. Treat the set of implemented hooks as part of the harness's interface, not an implementation detail.

Observe-only frameworks. Where native hooks are observe-only (e.g., LangChain, LlamaIndex), you cannot enforce a true fail-closed posture through the framework's callback event bus. Hard-code enforcement into synchronous wrapper classes around the underlying tool utilities or model clients rather than relying on the native listener surface.

Build it in, and log it

Hooks are an architectural decision, not a feature to bolt on later - the work is the same whether done calmly during initial development or urgently after an incident. Even if early policy is allow-all-with-logging, the hook points themselves should be in place from the first version. Map the work across the lifecycle: **Design** - define trust boundaries and the hook points each requires; **Implementation** - wire hooks into the harness execution loop; **Testing** - exercise critical disposition paths; **Pre-release review** - verify no sensitive action path bypasses a hook; **Deployment readiness** - confirm hook logs carry policy version, hook version, action metadata, and disposition reason codes.

Every hook invocation should emit a structured log entry (hook point, action intercepted, disposition returned, timestamp) - this produces the audit trail the logging section (3.3.3) requires and falls out of the architecture rather than needing separate instrumentation.

How sibling controls map to hooks

This section is the enforcement layer for controls defined elsewhere:

- **MCP hardening** (*6.4.2–6.4.4*): hash-pin verification, description-poisoning scan, and rug-pull revalidation fire at toolCallRequest.
- **Skill hardening** (*6.4.6*): signing, sandboxing, and permission-manifest controls fire at skillRegister / skillLoad; the digest pinned at skillRegister is what skillLoad verifies against.
- **Runtime guardrails** (3.3.2): input filtering at userMessage, output filtering at agentResponse.
- **HITL** (*3.1.5*): approval gates use the **Escalate** disposition at toolCallRequest.
- **Memory integrity** (*3.1.6*): read/write controls fire at memoryContextRetrieval and memoryStore.
- **Circuit breakers** (3.3.8): emergency halt uses **Deny** at whichever hook detects the anomaly.
- **Logging & monitoring** (3.3.3): structured trace data emits at every hook regardless of disposition.
- **Turn-scoped policies**: cumulative-taint tracking and per-turn rate limits enforce at turnStart / turnEnd.

See *Section 9* for hooks as runtime enforcement points.

3.3.8 Behavioral Circuit Breakers / Kill Switches

Detection without an automated response at agent speed is "just expensive logging." A behavioral circuit breaker is a real-time, in-line runtime policy engine that can interrupt a misaligned agent faster than a human could. It is not a binary on/off switch; it operates in graduated tiers (warn → limit → suspend → kill) and can act on an individual sub-agent or the whole system, notifying external agents/tools/MCP servers in use. Management can be centralized for coherence while enforcement stays decentralized for speed. Specify each circuit breaker against a concrete schema:

- **Policy:** the governing rule, baselined through behavioral testing / red-teaming before it is allowed to enforce (firewall-style), with support for manual exceptions and tuning. The policy must be hardened against tampering by the agentic system(s).
- **Signal:** what is monitored - e.g., output-guardrail violations, drift from the declared task goal, unexpected sub-agent spawning or loops, plan deviation, non-authoritative memory reads, anomalous tool-invocation patterns (fragmentation, ordering, sensitive domain/data combinations), tool schema/signature changes, privilege escalation, or system-file ops.
- **Trigger:** the condition that fires - e.g., a file write during an information-gathering phase, agent self-modification (acquiring new tools or changing its own config), reads of system directories when only project files were expected, or any tool schema/signature change.
- **Action:** the response - automatic tool revocation above an anomaly threshold, agent quarantine (retain observability while blocking external actions), halting the orchestrator, falling back to a safe read-only mode, or reducing the agent's action space rather than only stopping it.
- **Rollback:** use transaction boundaries, idempotency, and rollback where possible so an interruption can be undone; unsafe outcomes trigger containment plus rollback.
- **Audit / incident snapshot:** before a kill or rollback, capture a snapshot - freeze the context window and preserve the tamper-evident provenance chain (principals, decisions, tools, data accessed) so the audit log is a verifiable record rather than a trust assertion.

Per-component detection & action matrix. The signals and responses differ by where in the architecture the circuit breaker sits. Slice detection across the standard key components (KC1–KC6 from *Section 2.1*):

Key Component	Detection space (what to watch)	Killswitch actions
KC1 - LLM	Runtime output-guardrail violations (see 3.3.1); latent-space / hidden-activation monitoring; alignment of later messages to the initial task goal	Disable inference; quarantine session (auto-quarantine)
KC2 - Orchestrator	Sensitive core-agent logs; spawning sub-agents in an unexpected manner	Stop/disallow sub-agents; halt orchestrator (loop detector)
KC3 - Reasoning / Planning	Chain-of-thought keywords or concepts; plan deviation from declared intent	Restart plan; fall back to safe (read-only) mode (scope limiter)
KC4 - Memory	Any non-authoritative memory read (a taint for task reliability); bloat or unusual memory reads/writes; out-of-context queries (see 9.3)	Disable memory read/write; restrict read/write to non-sensitive data (circuit breaker)
KC5 - Tool harness / framework	Changes in tool descriptions or schemas; unusual tool-invocation patterns (unexpected fragmentation, combination, or ordering of tools; combinations of domains and data classifications, e.g. in email-tool parameters); cross-agent communication deviations	Revoke/suspend tool-call scope, permissions, rate, or allowed tools (especially web access, computer access, code execution) (rate & budget limiter)
KC6 - Operational environment	System-level logs; system-level file operations or privilege escalations (see 5.2–5.6)	Disable agent identity; treat outputs as compromised across storage, memory, tools (auto-quarantine)

Note: KC3 reasoning/planning signals are often monitored at the orchestration layer (KC2) where the plan is assembled and executed.

Worked trigger examples:

- If a protected task is in an information-gathering stage, a file *read* is expected, but a file *write* should trigger.
- If an agent write action is a self-modification (new tools or a change to the agent configuration) it should generally trigger - and the trajectory should be considered rogue, not merely blocked. Even where self-modification is accepted, the tool-mounting arguments must still be analyzed critically, as they may compromise the agent.
- If a protected task is expected to read business project files, then reading files in *system* directories should trigger.

- If recent history for a group of agents (sub-agents, or agents initiated by the same principal) shows a large rate of data-asset deletions (by quantity or size), the cross-agent pattern should trigger.
- If a sub-agent takes an action that increases access for a principal (e.g., approving an IT ticket granting HR-system access), a critical behavior to interrupt could be a *second* sub-agent updating that principal's email address to an external domain.
- If an LLM judge or graph-based anomaly detector finds an agent action attempting anomalous object mutations beyond the intended scope, trigger. A change to a tool schema/signature parameters must be blocked or inspected as suspect.

Common failure modes to design against: high-impact actions misclassified as low-risk; approval logic bypassed via an alternate workflow path; agents retrying failed actions and amplifying harm; irreversible actions with no rollback; and monitoring that only detects issues after the damage is done.

4. Enhanced Security Actions for Agentic AI Systems

Scope of this section. Generic controls that apply to *every* agentic system - authentication, authorization, data protection, code security, monitoring, logging, and prompt security - are the lifecycle backbone and live in *Section 3*. This section focuses on what differs by architecture: the *deltas* for single-agent, central-orchestrator, and swarm systems (orchestrator trust boundaries, inter-agent authentication, and swarm peer-to-peer communication). The single-agent baseline below (4.1) is retained from V1 as a reference checklist, but where it overlaps with Section 3 the Section 3 guidance is authoritative.

4.1 Single-Agent Systems

4.1.1 Authentication & Authorization

- **Implement OAuth 2.0/OIDC**
  - Use authorization code flow with PKCE for enhanced security
    - Use when a user is interacting with an agent/LLM through a native app or "one-page" web interface to prevent client secret leakage or token extraction.
    - Use when an agent is acting as a client to a web server without the possibility of a client secret.
  - Require explicit user consent through a consent screen showing specific permissions requested
  - Resource: Auth0 OAuth 2.0 Implementation Guide
- **Use managed identity services**
  - Configure cloud providers' identity services instead of embedding credentials
  - Use AWS IAM roles or Azure Managed Identities to avoid storing secrets
  - Resource (example): AWS IAM Roles for Amazon EC2
- **Apply Role-Based Access Control (RBAC)**
  - Define granular roles specific to agent functions, for example, only allow certain roles (user, dev, admin) to read/write system memory, etc.
  - Implement permission matrices to track role capabilities
  - Resource: NIST RBAC Standards
- **Grant only minimum necessary permissions**
  - Provide agents with only the permissions required for their specific tasks
  - Regularly audit and review permission assignments

- **Distinguish between read and write access**
  - Separate read and write permissions to minimize risk
  - Default to read-only access and explicitly grant write permissions only when needed
  - Resource: AWS IAM Policy Best Practices
- **Consider just-in-time credential issuance**
  - Implement temporary credentials with limited scope and lifetime
  - For managing cross-domain authentication, apply the Token Vault pattern described in Section 3.1.8.
  - Credentials only when needed and ensure they expire quickly
  - Resource: HashiCorp Vault Secrets Management

4.1.2 Data Protection

- **Encrypt sensitive data**
  - Use strong encryption algorithms for data at rest and in transit
  - Implement TLS 1.2+ for transit, AES-256-GCM for storage
  - Resource: NIST Cryptographic Standards
- **Implement Data Loss Prevention (DLP)**
  - Deploy DLP solutions to monitor and prevent leakage of sensitive information
  - Set up scanning of data inputs/outputs to detect PII, credentials, or other sensitive data
  - Resource: Open Source DLP Solution - OpenDLP
- **Use data classification and sensitivity labels**
  - Apply labels that guide AI agent's access and handling of data
  - Data is accessed deterministically and then compared to what kind of data access level the agent has.
    - § For example HIPAA data cannot be accessed by a non HIPAA-compliant agent
  - Create tiered classifications like public, internal, confidential, and restricted
  - Resource: Microsoft Information Protection
- **Apply data minimization principles**
  - Collect and process only the data necessary for the specific task
  - Create data flow diagrams to identify unnecessary data collection
  - Resource: GDPR Data Minimization Principle

4.1.3 Code Security

- **Establish automated testing pipelines**
  - Set up CI/CD pipelines with integrated security scanning
  - Include SAST, DAST, and SCA tools in the pipeline
  - Resource: OWASP DevSecOps Guideline
- **Conduct thorough code reviews**

  - Implement both automated and manual code reviews before deployment
  - Use checklists specific to AI system security concerns
  - Resource: OWASP Code Review Guide
- **Monitor dependencies**
  - Continuously check external libraries for vulnerabilities
  - Set up automated dependency scanning and alerts
  - Resource: GitHub Dependabot

4.1.4 Monitoring & Incident Response

- **Log all actions and establish baselines**
  - Record comprehensive logs of all agent activities
  - Establish behavioral patterns to detect anomalies
  - Maintain a log chain of thought for particularly sensitive operations. These logs should be assigned the same data classification as the highest level of data involved in their generation.
  - Resource: Cloud Security Alliance - Security Logging Guidelines
- **Implement real-time anomaly detection**
  - Use machine learning to detect unusual behavior patterns (using an ML enabled SIEM tool)
  - Deploy systems that can identify deviations from normal operation
  - Resource: Awesome Anomaly Detection
- **Set up alerts for suspicious events**
  - Configure notifications for unusual patterns or security concerns
  - Alert on access attempts to sensitive endpoints, spikes in requests, unusual IP origins
  - Resource: PagerDuty Incident Response Documentation
- **Develop incident response plans**
  - Create detailed procedures for addressing security breaches
  - Include containment, investigation, remediation, and recovery steps
  - Resource: NIST Computer Security Incident Handling Guide
- **Build emergency off-switches**
  - Implement kill switches to immediately revoke access privileges
  - Create automated and manual mechanisms to stop agent operations
  - Resource: Anthropic AI Safety Research

4.1.5 Prompt Security

- **Implement input validation**
  - Filter user inputs using rule-based patterns and NLP techniques or filtering for multi-modal processing
  - Check for malicious patterns before processing by the AI system (WAF Rules)

    - Resource: Guide to LLM Prompt Injection Defenses
- **Apply content filtering on AI outputs**
    - Screen AI-generated responses for inappropriate or harmful content
    - Use both pattern matching and ML-based content classifiers
    - Resource: Perspective API for Content Moderation
- **Harden system prompts**
    - Embed security policies directly into the AI's foundational instructions
    - Structure prompts to reject harmful requests and maintain guardrails
    - Resource: OpenAI System Message Guidelines
- **Sanitize all inputs**
    - Clean and normalize inputs to prevent injection attacks
    - Remove or escape special characters and formatting that could alter behavior
    - Resource: OWASP Input Validation Cheat Sheet

4.2 Multi-Agent Systems with Central Orchestrator

Multi Agent: Multiple intelligent agents that interact with each other and a shared environment to solve problems or achieve goals. The agents can operate independently, discern their surroundings, make decisions, and act. The central orchestrator is responsible for coordinating the agents' interactions, assigning tasks, and maintaining the overall workflow or state.

4.2.1 Authentication & Authorization

- Use authentication and authorization controls from 4.1.1
- **Establish separation of control planes**
    - Create clear separation between different agent functionalities
    - Utilize the user assumption architecture pattern from section 2.2.1 to enforce user permissions on an agent
    - Maintain distinct permission sets for each agent based on least privilege
    - Resource: AWS Separation of Control and Data Planes
- Authenticate and verify all agent interactions (see section 4.2.3)
- **Restrict discovery to known and trusted servers**
    - Hard code discovery servers in agent instructions
    - Prevent agents from communicating with discovery servers using network and logical controls

4.2.2 Orchestrator Security

- **Harden the central orchestrator's API**
    - Implement robust authentication and authorization mechanisms
    - Use rate limiting, input validation, and detailed logging

  - Resource: OWASP API Security Top 10
- **Protect against control-flow hijacking**
  - Validate agent responses to prevent manipulation of the orchestrator's decisions
  - Implement integrity checks on operational metadata and agent outputs
  - Resource: MITRE ATLAS Framework for AI Threats
- **Mitigate the "confused deputy problem"**
  - Prevent scenarios where trusted components are tricked into performing unauthorized actions
  - Implement additional context validation and cross-check requests
  - Resource: The Confused Deputy Problem
- **Formalize On-Behalf-Of (OBO) delegation and intent binding**
  - Delegate user authority via OAuth 2.0 Token Exchange (RFC 8693), never raw user credentials. Issue short-lived, task-bound, audience-bound tokens; each sub-agent or intermediary performs its own exchange.
  - Enforce delegation depth limits and monotonic scope attenuation: each hop's token is a proper subset of its parent's, with decaying TTL and cascade revocation.
  - Bind authorization to the declared user intent for the current task, and deny actions outside that intent even when permission and capability would otherwise allow them. Have the orchestrator pass a signed, immutable, intent-locked task descriptor that sub-agents cannot rewrite mid-chain. This is the structural mitigation for the confused deputy problem.
  - For MCP/OAuth proxy architectures, avoid static client IDs (vulnerable to consent-cookie reuse via dynamically registered malicious clients): use per-client consent storage and an MCP-level consent UI with CSRF protection, and prohibit token passthrough.

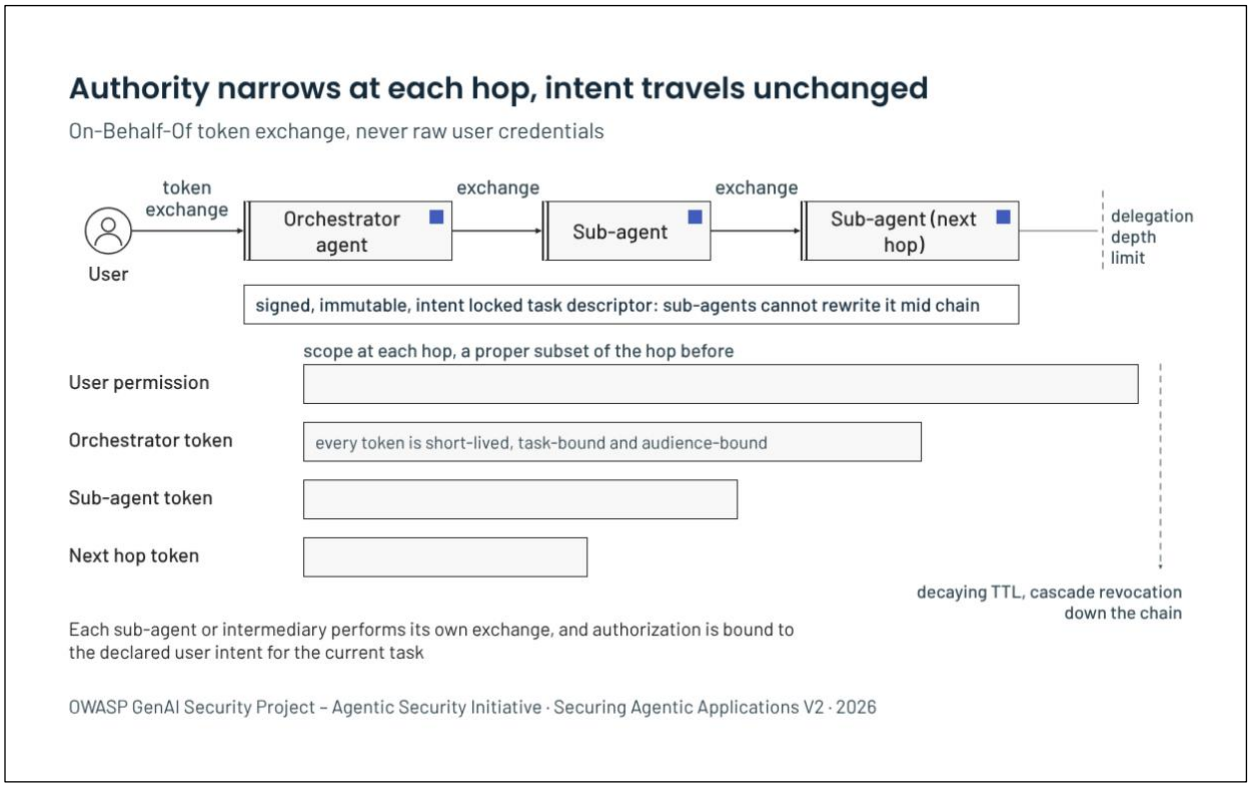


Figure 4.1: On-Behalf-Of delegation from the user through the orchestrator to each sub-agent. Every hop performs its own OAuth 2.0 Token Exchange and receives a short-lived, task-bound, audience-bound token whose scope is a proper subset of its parent's, with decaying TTL, cascade revocation and a delegation depth limit. The signed, immutable, intent-locked task descriptor travels with the chain unchanged. The narrowing is enforced by authorization server policy at each exchange.

4.2.3 Inter-Agent Communication

- **Implement secure communication protocols**
  - Use protocols with strong encryption for all agent interactions
  - Implement mutual TLS (mTLS) for two-way authentication
  - Resource: Mutual TLS (mTLS) Authentication
- **Verify the identity of each communicating agent**
  - Use certificate-based or token-based authentication mechanisms
  - Resource: JWT Authentication Best Practices
- **Use secure message queuing systems**
  - Implement message brokers with security features for asynchronous communication
  - Use systems like RabbitMQ, Kafka, or NATS with authentication and encryption
  - Resource: RabbitMQ Security
- **Deploy Policy Enforcement Points (PEPs)**
  - Add security checkpoints to communication channels for policy enforcement

  - Implement as middleware or service proxies that validate each interaction
- **Apply rate limiting for agent interactions**
  - Prevent abuse by limiting the frequency of requests between agents
  - Implement exponential backoff for failed requests
  - Resource: GitHub API Rate Limiting

4.2.4 Trust Boundaries

- **Apply Zero Trust security principles**
  - Implement "never trust, always verify" approach across the architecture
  - Verify every access request regardless of source
  - Resource: NIST SP 800-207 Zero Trust Architecture
- **Implement network segmentation and agent isolation**
  - Create distinct security zones within the system
  - Limit interconnectedness of agents to contain potential breaches
  - Resource: NSA Network Segmentation Guide
- **Use containerization technologies**
  - Sandbox individual agents using Docker or similar technologies
  - Restrict access to underlying OS and system resources
  - Resource: Docker Security Best Practices

4.3 Multi-Agent Systems with Swarm Architecture

4.3.1 Authentication & Authorization

- Use authentication and authorization controls from 4.1.1 and 4.2.1
- **Identify trusted agents and actions**
  - Swarm systems don't necessarily have an easy guide post for the correct action and trusted collaborators. Assign this trust and action outside the agentic flow
  - Limit or prevent the swarm from adding new agents without human guidance.

4.3.2 Decentralized Identity & Trust

- **Implement Decentralized Identifiers (DIDs)**
  - Use W3C standard DIDs for self-sovereign agent identities
  - Enable verification without central authority
  - Resource: W3C Decentralized Identifiers Specification
- **Use Verifiable Credentials (VCs)**
  - Implement VCs for agents to prove specific attributes
  - Enable secure and verifiable proofs of capabilities or authorizations

    - Resource: W3C Verifiable Credentials Data Model
- **Create decentralized reputation systems**
    - Build systems where agents develop reputation scores based on behavior
    - Use these scores to make trust decisions for collaboration
    - Resource: The OpenCerts Framework for Verifiable Claims
- **Guard against Sybil-inflated reputation**
    - Cheap-to-mint identities can mutually vouch to inflate a malicious agent's score faster than genuine behavior would earn it
    - Pair reputation with a minting-cost control: staking, human-sponsored first credential, or rate-limited DID registration

How?

- **Agent Identity:** Each AI agent can have its own DID, providing a persistent, verifiable identity across different interactions and environments.
- **Credential-Based Trust:** Verifiable Credentials can establish an agent's capabilities, permissions, or trust level within the system.
- **Inter-Agent Authentication:** Agents can verify each other's identities and credentials without centralized coordination.
- **Provenance Tracking:** DIDs can help track the origin of information or actions in a multi-agent system, establishing clear provenance.
- **Access Control:** VCs can determine which agents have access to specific resources or capabilities within the system.
- **Role Specialization:** Credentials can formalize specialized roles among different agents (e.g., reasoning agent, retrieval agent, etc.).

4.3.3 Secure Peer-to-Peer Communication

- **Select appropriate P2P protocols**
    - Choose protocols based on specific security and performance needs
    - Consider TLS/SSL, DTLS, Noise Protocol, or specialized frameworks
    - Resource: Noise Protocol Framework Specification
- **Ensure both encryption and authentication**
    - Implement strong encryption to protect confidentiality
    - Use robust authentication to verify identity of participating agents
    - Resource: NIST SP 800-175B: Guideline for Using Cryptographic Standards

4.4 Cross-Architecture Security Considerations

- **Secure API Design for Agent Communication**
  - Implement standardized interfaces with comprehensive input validation. (See Appendix E for protocols/standards)
  - Apply rate limiting and authentication regardless of architecture
  - Resource: OWASP API Security Top 10
- **Continuous Security Testing**
  - Regularly scan all agent architectures for vulnerabilities
  - Implement automated and manual security testing processes
  - Resource: NIST Security Testing Guidelines
- **Security Monitoring and Threat Detection**
  - Implement comprehensive monitoring regardless of architecture
  - Adjust detection strategies based on architectural specifics
  - Resource: MITRE ATT&CK Framework
- **Secure Agent Updates and Maintenance**
  - Implement secure update mechanisms for all architectures
  - Verify integrity of updates before deployment
  - Resource: NIST Secure Software Development Framework

This comprehensive list provides developers with practical security actions covering all major aspects of agentic AI system security across different architectures, along with relevant web resources for further reading.

5. Key Operational Capabilities

Connecting Agentic AI applications with other systems or environments such as APIs, databases, code interpreters, web browsers, or the operating system enables great functionality (skills). However, each new operational connection poses unique security concerns, ranging from data breaches and illegal actions to system compromise. The following listed controls are necessary to reduce these inherent risks.

5.0 Capability Control Matrix

Before the per-capability controls below, classify each operational capability the agent holds along the dimensions in the matrix. This makes the capability-composition gate from *2.7.3* (Lethal Trifecta / Rule of Two) concrete: if a single agent's capabilities together cover *untrusted input + sensitive data + external communication*, the design must add sandboxing, HITL, runtime monitoring, or architectural separation - or remove one leg.

Capability (KC6)	**Capability class**	**Touches sensitive data?**	**Consumes untrusted input?**	**Can communicate externally / change state?**	**Required controls**	**Escalation trigger**
5.1 API Access	read-only → high-impact	Often	Via responses	Yes	Scoped OAuth/keys, allow/deny lists, templates, schema validation, rate limit, logging	Write/admin scope; new destination domain
5.2 Code Execution	high-impact	Possibly	Yes (generated code)	Yes	Mandatory sandbox, resource limits, static analysis, command allowlist, HITL for high-risk	Network/file access from sandbox; arbitrary code

5.3 Database / RAG	bounded-write → high-impact	Yes	Via retrieved content	Bounded	Parameterized queries, least-privilege accounts, post-retrieval PII filter, RBAC vector DB	CRUD/DDL access; cross-tenant retrieval
5.4 Web Use	read-only → bounded-write	Via browser data	Yes (web content)	Yes	Sandboxed browser, URL allowlist, SSRF segmentation, content sanitization, logging	Authenticated session reuse; internal-network reach
5.5 PC / Computer Use	high-impact	Yes	Yes (files, screen)	Yes	OS sandbox, path/command allowlists, dry-run, HITL, screenshot redaction, repo isolation	OS command exec; destructive git/file ops
5.6 Critical Systems	high-impact	Yes	Possibly	Yes	Air-gap/segmentation, MFA, mandatory HITL, read-only default, physical interlocks	Any control/write action

For each capability also record the **required telemetry** (what must be logged) and an **escalation trigger** (the condition that forces HITL, throttling, or a circuit-breaker action - see *3.3.8*).

- **Trigger:** A declared effect annotation moving in the permissive direction (e.g., destructiveHint changing from true to false).
- **Binding Requirement:** When a gate consumes an effect annotation, bind the declaration to the contract it was declared against. Treat an absent annotation as unobserved rather than safe.

5.1 KC6.1 – API Access

Core Threats: Unauthorized data access/leakage (T3), API abuse (DoS, cost overruns)(T4, T2), Compromised keys (T3, T9)

Controls:

1. **Enforce Least Privilege with Fine-Grained OAuth Scopes or Limited API Keys**
    - Define the minimum necessary activities for task completion (e.g. never grant an admin's complete access to an agent)
    - Grant access solely to essential assets with minimal access scope / issue access tokens only to necessary clients/agents
    - Implementation Options:
        - OAuth 2.0: Establish specific OAuth scopes using services like: Okta, Auth0, Keycloak, Cloud provider integrated services
        - SDKs: Utilize Authlib or Passport.js to streamline OAuth operations
        - API Keys: Employ provider features to enforce limits:
            - AWS API Gateway allows restriction to specific API stages/methods
            - Configure IP address restrictions or read-only access where available
        - Cloud IAM: For agents operating as cloud services (e.g., AWS Lambda, Google Cloud Functions), assign specific IAM roles with restricted permissions

2. **Set up Allow/Deny Lists for APIs**
    - Prevent manipulation via prompt injection or compromised tools
    - Clearly define authorized domains or URL pathways
    - Implementation Approaches:
        - Configure a list of permitted base URLs (Allow-List) or URL patterns
        - Authenticate target URLs against this list before executing external requests
        - Use outward proxy or API Gateway to permit connections only to pre-approved destinations
        - For containerized agents, implement network policies to limit outgoing traffic

3. **Prefer API Templates Over Complete LLM-Generated API Calls**
    - Mitigate risks of unsafe or malformed API calls
    - Improve reliability by hard coding unchanging/common fields

4. **Best Practices:**
    - Use prepared templates where LLMs only populate established parameters
    - Utilize templating libraries like Jinja2
    - Leverage structured output capabilities with strictly defined schemas
    - Define parameter types and requirements before sending requests

- Sanitize web-fetched content using tools like DOMPurify
- Implement Content Security Policy (CSP) when rendering content

5.2 KC6.2 – Code Execution

Core Threats: Arbitrary code execution (RCE) (T11, T3), Code injection (T11, T2), DoS via resource exhaustion (T4), Leakage of confidential information (T6)

Controls:

1. **Implement Mandatory Sandboxing**
   - OS-level isolation, containers, VMs, WebAssembly, or cloud solutions
   - Reference implementations:
     - NVIDIA's WebAssembly Sandboxing
     - LangChain's E2B Data Analysis

2. **Perform Static Analysis on Agent-Generated Code**
   - Use tools like:
     - Bandit for Python
     - Semgrep for multi-language analysis
     - CodeShield for LLM-generated code

3. **Enforce Resource Limitations**
   - Set strict CPU, memory, and execution time limits
   - Implement timeouts for all operations

4. **Additional Security Measures**
   - Use code signing or verification when possible
   - Implement strict file system and network restrictions
   - Use allow-lists for permitted commands
   - Require Human-In-The-Loop (HITL) approval for high-risk operations
   - Sanitize all executed code output
   - Implement runtime monitoring during execution

5.3 KC6.3 – Database Execution (Queries and RAG access)

Core Threats: SQL injection (T2), Unauthorized data exposure/modification (T3), RAG data poisoning/sensitive data retrieval (T1, T5, T3)

Controls:

1. **Use Managed Vector Databases with Access Controls**
    - Restrict data ingest to match the access control capability (never ingest data that can't be protected from unauthorized users)
    - Apply data classification labels to enforce user privilege
        - Utilize specialized RBAC RAGs like Elastic RAG
        - Pinecone with authentication platforms like Clerk
        - Authorization platforms like Aserto

2. **Implement Query Safety Measures**
    - Use parameterized or template queries/ORMs exclusively. Never construct SQL using string concatenation.
    - Grant database access via least-privilege user accounts, e.g. SELECT only, limited tables, or row/column level security
    - Filter/block dangerous SQL keywords (e.g., DROP, TRUNCATE)
    - Validate all user inputs influencing queries - if possible, derive from controlled channels only vs context window.

3. **RAG-Specific Controls**
    - Implement post-retrieval filtering to check for sensitive content (PII) before agent use
    - Apply content verification before embedding into vector stores
    - Rate-limit retrieval operations

4. **Limitation (Excessive Retrieval)**

5.4 KC6.4 – Web Use

Core Threats: Malicious web content (XSS, exploits)(T11), Leakage of confidential information (T6), Phishing/deception (T7), Server-Side Request Forgery (SSRF)(T2, T3), Access to user's browser data - Sensitive Data Exposure (T3)

Controls:

1. **Sandbox Browser Components**
   - Run in tightly controlled environments
   - Avoid using extensions or operating directly in the user's browser

2. **Implement URL Security**
   - Use URL filtering with allow lists
   - Leverage Secure Web Gateway (SWG) or Data Loss Prevention (DLP)
   - Perform reputation checks on target domains
   - Enforce proper TLS verification
   - Detect and block open redirects or excessive redirection chains. Further, use DNS-based filtering in addition to HTTP-level filtering.

3. **Apply Protective Measures**
   - Use network segmentation to prevent access to internal systems (mitigating SSRF)
   - Throttle web access rate to prevent abuse
   - Restrict downloadable file types
   - Implement content sanitization before processing
   - Inspect embedded/linked content (e.g., iframes, JS-injected sources)

4. **Logging and Monitoring**
   - Log all URL accesses and HTTP responses

5.5 KC6.5 – Controlling PC Operations (Filesystem, OS Commands)

Core Threats: Unauthorized file access/modification (T3), Arbitrary OS command execution (T11, T3, T2), Lateral movement (T3)

Controls:

1. **Apply Strict Isolation**
   - Use OS-level sandboxing
   - Run agents as restricted users with minimal privileges
   - Consider dedicated containers or VMs for isolation

2. **Implement Access Controls**
   - Define strict allow/deny lists for permitted OS operations and file paths
   - Use virtual filesystem interfaces where appropriate
   - Intercept OS command calls to enforce allow-lists

3. **Limit accumulation of sensitive data**
   - To maintain its status, sandbox should not accumulate data and sessions

4. **Enhance Monitoring and Oversight**
   - Log all OS-level actions performed by the agent
   - Consider HITL approval for critical operations
   - Implement command validation before execution

5.5.1 Coding & Computer-Use Agent Guardrails

Coding agents and "computer use" agents are, by definition, a Lethal Trifecta (see *2.7.3*): they process untrusted input (repositories, issues, web pages, screens), hold access to sensitive data (credentials, source, git, CI/CD, authenticated sessions), and can communicate externally. They therefore require the controls below in addition to the general KC6.5 controls.

- **Treat repository-level configuration as an untrusted, execution-layer supply-chain vector.** Files such as CLAUDE.md, .cursorrules, .github/copilot-instructions.md, mcp.json, and .cursorignore travel inside cloned/forked/shared repos and can control agent behavior (system prompts, allowed tools, MCP connections) before any trust dialog. Documented cases include a repo settings file redirecting the model API endpoint to an attacker (capturing requests/credentials) and a changed .cursorignore exposing credential files. Treat user-level config as trusted and repo-level config as untrusted; require a user approval gate before repo-provided instructions or repo-controlled config can execute; and integrity-check MCP server configs.

- **Command permission model:** default-deny plus an allowlist. Make allow/deny lists for commands, file paths, and network operator-configurable (not only developer-set). Validate that a command is grounded in session intent before running it, and escalate to HITL rather than blanket-blocking the session. Use risk-based HITL to avoid approval fatigue.
- **Restricted git / destructive-operation safety:** prevent destructive operations (e.g., force push, reset --hard, DROP TABLE), protect branch policies, and require commit signing.
- **Path & credential hygiene:** use .ignore-style configs to keep credentials out of context and long-term memory; detect "toxic combinations" (sensitive-file access plus an external tool call in one session); validate arguments for directory traversal and symlinks before high-privilege writes.
- **Repo isolation / sandboxing:** clone, read, and modify repositories inside an OS- or cloud-VM-isolated sandbox (kernel-isolated filesystem and execution, restricted inbound/outbound network); make tools "sandbox-aware"; define sub-agent / worktree permission-inheritance rules.
- **Session isolation for browser/computer use:** use dedicated browser profiles rather than the user's authenticated sessions, and redact screenshots / sensitive on-screen output.
- **Dry-run / preview** risky operations before committing them, and intercept agent-to-LLM and agent-to-tool I/O (via hooks, see *3.3.7*) to scan for jailbreaks, sensitive data, malicious code/URLs, and drift between an action and prior intent.

5.6 KC6.6 – Operating Critical Systems (e.g., SCADA controls)

Core Threats: Catastrophic physical outcomes (T2, T3, T6, T7), Malicious control injection (T2, T6), False data influencing decisions (T1, T5)

Controls:

1. **Apply Maximum Isolation**
   - Use air-gapped or highly segmented networks
   - Implement physical security measures

2. **Enforce Multi-Factor Authentication**
   - Switch to external flow to Require MFA for specific sensitive actions where possible - do not use agent to mediate MFA flow or retain privileged session
   - Implement mandatory HITL for every critical action, succinct and engaging enough to retain the user's attention each time (excessive/inefficient prompting results in indiscriminate approval)

3. **Implement Safety Mechanisms**
   - Default agent permissions to read-only monitoring
   - Use anomaly detection tuned to system safety parameters
   - Adhere to industrial control security standards
   - Ensure physical interlocks and emergency overrides exist

6. Agentic AI and the Supply Chain

6.1 Code Security

6.1.1 - Third party libraries & frameworks

- Standard supply chain applies
- SCA scanning
- Hashes & package locks for 3rd party versioning
- SBOMs
- Licenses & provenance should be validated for third party libraries before install

6.1.2 - LLM/Agent generated code

- Utilize sandboxing for any agent generated/run code (3.2.3)
- Validate packages before running generated code
- HITL for high-risk code execution operations

6.2 Environments & Development

6.2.1 - LLM / Logic System

- Use the same LLMs in lower environments & production for higher reliability
- Ensure the provenance of LLMs in a public registry (like Hugging Face) by verifying hashes and contributors
- Use LLM and SCA scanners to scan for both third-party package vulnerabilities and security issues within LLM code (where applicable)

6.2.2 - Version Control & Code Management

- Utilize version control with LLMs & logic systems to track behavior
- Implement version control on prompts & instructions to increase reliability and auditability of behavior
- Use commit IDs and hashes for data changes

6.2.3 - Permission management

- Don't give agents access to the repositories/data sources which build/deploy them
- Manage permissions to data sources outside the environments the agent is running in
- Use code to modify IAM/roles permissions for agents to review changes

6.3 Agent & Tool Discovery

6.3.1 - Agent Cards

- Identify trust relationships with agent cards and descriptions, and minimize trust with unvalidated agent systems
- Employ DIDs for agent verification and authentication
- Apply the tool-definition integrity controls in 6.4.2 to agent cards: pin the card by digest at first trust, re-fetch and re-compare on a defined schedule, and treat a changed card as a re-approval trigger rather than a silent update.

6.3.2 - Local vs Remote Agent Registries

- Use logical or network barrier between local & remote registries to prevent registry confusion and accidental agent boundary traversal
- Classify and separate private and public agents, data and actions. Prevent agents from sharing data or performing actions across these boundaries
- Use trust relationships and controls like certificate pinning to ensure agents only communicate with approved registries and environments

6.4 Tool Lifecycle & MCP-Specific Supply-Chain Controls

This section covers the supply chain *for the tools an agent uses* - including MCP servers and Agent Skills. The goal is actionable build/runtime control, not an AI-BOM or compliance artifact. MCP is treated as a tool-integration profile of KC5 (see *2.1 note under KC5.3*); the controls below are where its hardening requirements live.

6.4.1 Standardized Provenance Record

To ensure consistency across tool, skill, and memory auditing, all components must implement a unified Provenance Record schema. This allows runtime enforcement and monitoring logic to treat trust decisions uniformly - for example:

- **source_id**: Unique identifier for the origin (e.g., server URI, repository URL).

- **author**: Verified publisher or entity identity (e.g., DID, signing principal).
- **timestamp**: ISO-8601 record of when the artifact was created or captured.
- **trust_level**: Categorical classification (e.g., System, Vetted, User, Untrusted).
- **integrity_hash**: Cryptographic digest (SHA-256 or better) of the artifact content.
- **last_verified_at**: Timestamp of the most recent validation check against the source.

6.4.2 - Tool onboarding & catalog governance

- Maintain an internal **vetted registry** of approved tools / MCP servers; block direct pulls from public repositories in production. Tool/server "squatting" and "slopsquatting" (names a model is likely to hallucinate or suggest) make open registries an active attack surface.
- For each server/tool onboarded: SAST-scan its code, produce an SBOM of its dependencies, and run it in a behavioral sandbox to verify it performs only its advertised function. Verify publisher identity and prefer canonical sources.
- Require a security-review admission gate before any new tool is added; test tool updates in staging before production; actively decommission unused tools.
- Maintain a **signed, version-pinned catalog manifest** (server URI, tool name, approval timestamp) signed with a key held outside the agent runtime and stored in version control. Each entry must adhere to the *Standardized Provenance Record (6.4.0)*. An unsigned or unverifiable manifest is a deployment-gate failure.

6.4.3 - Tool definition integrity (anti "rug-pull")

- **Pin tool descriptions and schemas by cryptographic hash** at first registration. Re-hash on every session start and on every catalog refresh, and compare against the pinned values; block execution when a live definition no longer matches its approved digest.
- A hash mismatch blocks the tool pending operator re-approval, with an alert that includes the diff. Define explicit re-approval triggers: a change to a tool's name, schema, or description; a change to the server's TLS certificate; the server hostname resolving to a new address; or a changed tool list.
- Never carry "approved" status across session boundaries ("approve once, trust forever" is the vulnerability). Apply a maximum **Approval TTL**; expiry triggers automatic re-validation, not silent re-approval. Record, alongside every pinned digest, the observation timestamp and the age of the source snapshot at the time of approval. An approval is only as fresh as the input it was computed against. Audit staleness arises from two independent producers: subject drift after judgment and instrument lag at judgment. Digest pinning models only the first; instrument lag - where the evaluation timestamp postdates the last day the judged text was published - emains invisible to models assuming the scanner sees the present.
- **Catalog-drift detection:** alert when a server adds, removes, renames, or changes tools even if no individual approved definition changed, and keep an append-only catalog audit log.

6.4.4 - Treat tool descriptions and responses as adversarial input

- Scan every tool name, description, and input/output schema for hidden-instruction patterns before admitting it to context (Tool Poisoning); reject tools that fail the scan. A pattern baseline includes injection delimiters (<IMPORTANT>, IGNORE PREVIOUS), cross-tool-orchestration phrases ("before using this tool..."), credential/path references, exfiltration patterns, base64 blobs, and zero-width Unicode. Render the full description to the operator at registration, not just the tool name.
- Enforce strict isolation between tool descriptions: a description must not reference, modify, or condition behavior on other tools (Cross-Tool Shadowing).
- Validate every tool **response** against the approved schema before it enters the model context; a non-conforming response must not enter context. MCP tool responses are a primary indirect-prompt-injection surface.
- Pattern/regex sanitization is insufficient as the *primary* defense (fragmented multi-layer payloads evade filters and reassemble at the sink). **Prioritize schema enforcement and runtime isolation over content sanitization.**

6.4.5 - Asset Access Control Matrix

To prevent unauthorized data exfiltration and configuration hijacking, access to sensitive security assets must be strictly policed. The following matrix defines the authoritative read/write boundaries for credentials, configuration files, and identity assets across the tool and skill lifecycle. Any attempt to deviate from these boundaries must trigger a behavioral circuit breaker (see 3.3.8).

Asset Type	Credential Assets	Configuration Assets	Identity Assets
Source / Entity	*e.g., API keys, OAuth tokens, JIT credentials*	*e.g., mcp.json, model config, skill manifests*	*e.g., DIDs, signing keys, service accounts*
MCP Servers / Tools	**Read:** Ephemeral/Scoped only. **Write:** PROHIBITED.	**Read:** Own config only. **Write:** PROHIBITED (Rug-pull protection).	**Read:** PROHIBITED. **Write:** PROHIBITED.
Agent Skills (Packages)	**Read:** Per-manifest scope. **Write:** PROHIBITED.	**Read:** Sandbox-scoped. **Write:** Signed updates only.	**Read:** PROHIBITED. **Write:** Prohibited outside registry.
Model / LLM Context	**Read:** Masked/Redacted. **Write:** PROHIBITED.	**Read:** Masked/Redacted. **Write:** PROHIBITED.	**Read:** Public metadata only. **Write:** PROHIBITED.

Asset Type	Credential Assets	Configuration Assets	Identity Assets
Orchestrator / Proxy	**Read:** Full (Vault-backed). **Write:** Full (Audit-logged).	**Read:** Full (Hash-pinned). **Write:** Admin-only.	**Read:** Internal Registry. **Write:** Signing service only.

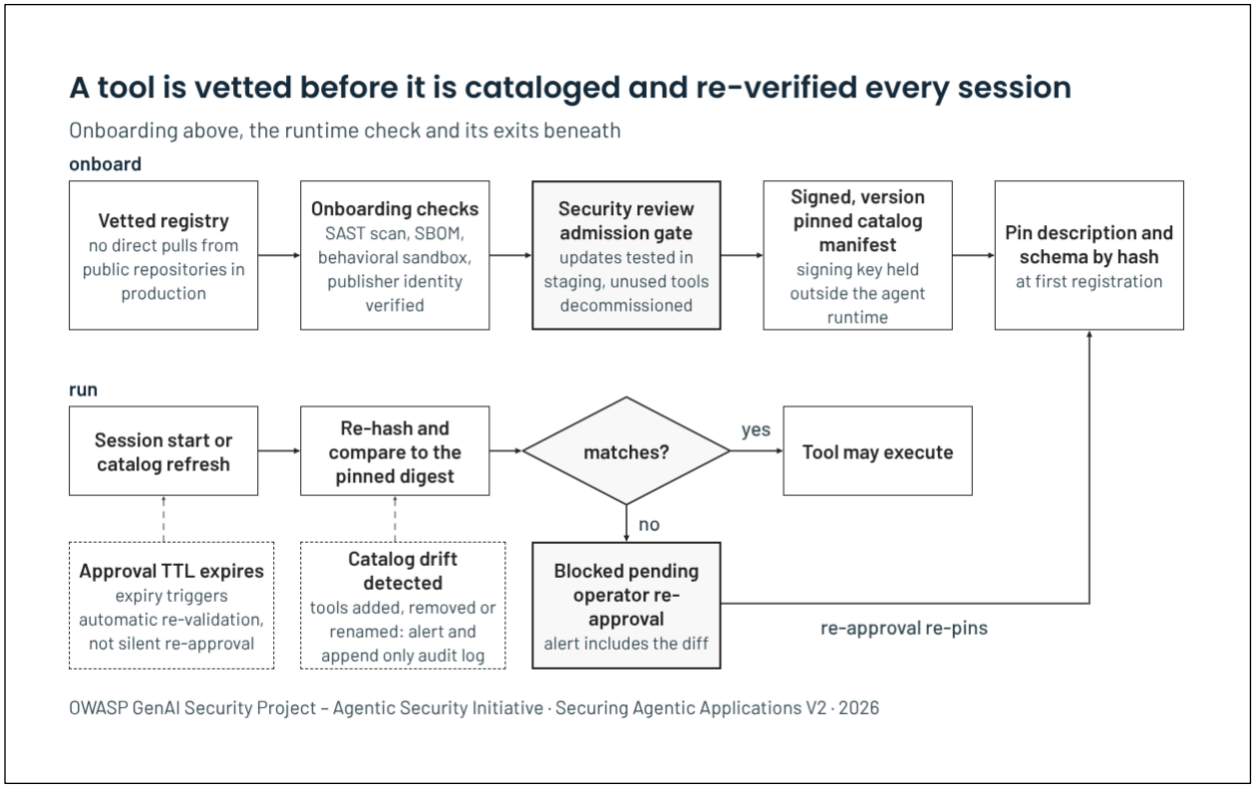


Figure 6.1: The tool lifecycle. Onboarding runs from a vetted registry through SAST scan, SBOM and behavioral sandbox checks, a security review admission gate and a signed, version pinned catalog manifest to pinning each tool's description and schema by hash. At every session start or catalog refresh the definition is re-hashed and compared to the pinned digest; a mismatch blocks the tool pending operator re-approval. Approval TTL expiry forces automatic re-validation, and catalog drift raises an alert and an append only audit log entry.

7. Assuring Agentic Applications

Assurance strategies for agentic applications must extend beyond static testing. Consider these approaches:

> **Scope of this section.** This guide's scope excludes red-teaming *playbooks*; detailed red-team methodology, attack recipes, and benchmark-tool catalogs belong to the separate **OWASP ASI Red Teaming Guide** and are referenced rather than reproduced here. The focus of this section is **lifecycle validation**: tying assurance back to the system's intended scope, assumptions, threat model, controls, behavioral tests, runtime monitoring, and regression triggers. The red-teaming material retained below (7.1) is a brief orientation and pointer to that guide.

7.0 Assurance as Lifecycle Validation

This chapter follows the agentic security lifecycle in two halves. The first half is **design-time assurance**: the architectural and configuration choices that bound what an agent can do before it ever runs. The second half is **validation that the design holds**, covered through red teaming (7.1) and behavioral testing (7.2). The ordering is deliberate - testing has nothing to falsify until the design has named its own assumptions.

7.0.1 Assurance objective

The objective is to determine whether an agentic application is sufficiently constrained, tested, and monitored to operate within its intended scope. Assurance does not mean proving an application is perfectly safe; it means establishing a **risk-based, evidence-backed case** that the application's design assumptions hold under realistic and adversarial conditions. It applies familiar software-assurance practices to agentic systems - requirements traceability, control validation, regression testing, release evidence, change-impact analysis, and production monitoring: define what the system is supposed to do, identify what it must not do, implement controls, test whether those controls hold, and revalidate the affected parts when behavior-defining elements change.

The assurance scope is the *actual application being assessed* - its purpose, trust boundaries, data flows, data access, tools, memory, orchestration, identity model, external integrations, deployment context, and operational controls - not a generic agent pattern. This holds whether the application is a simple single-agent workflow or a complex system with delegation, tool chains, retrieval, long-running state, or external protocols.

The chain that traces design intent to runtime evidence:

design scope → assumptions → threat model → controls → validation tests → runtime monitoring → regression triggers.

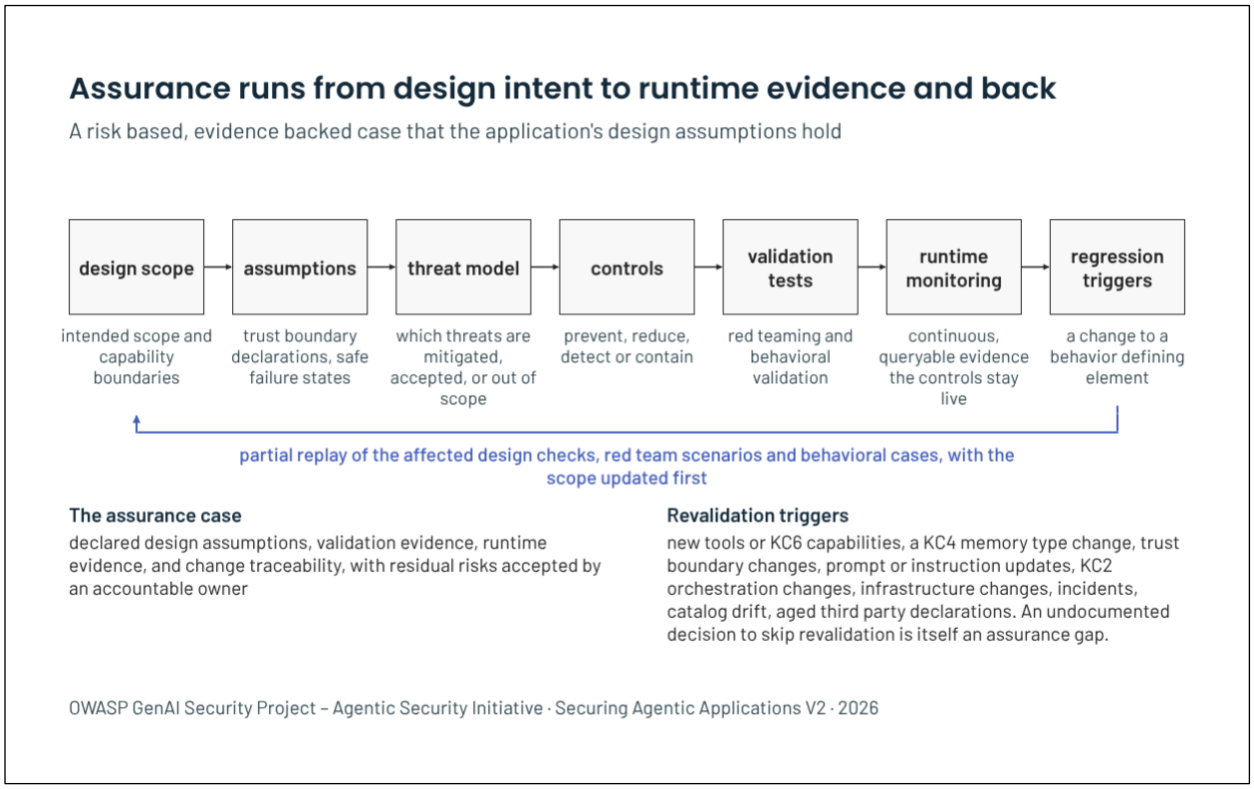


Figure 7.1: The assurance chain, design scope, assumptions, threat model, controls, validation tests, runtime monitoring and regression triggers, with the loop a trigger sends back to the scope: a partial replay of the affected design checks, red team scenarios and behavioral cases, with the scope updated first. An undocumented decision to skip revalidation is itself an assurance gap.

7.0.2 Definitions used in this chapter

- **Intended scope** - what the application is *for*: its purpose, the behavior it should and should not exhibit, the data and tools it may use, and the trust boundaries within which these hold. It must be precise: a scope stated vaguely ("act as a helpful assistant") cannot be configured, tested, or monitored against. For any given action a reviewer must be able to decide whether it falls within the boundary - including capability *compositions* that remain out of scope even where each capability is individually permitted (see *2.7.3*).
- **Behavior-defining element** - any configurable element of the deployed application whose modification can change behavior in a way that may invalidate a prior design assumption, control, or test result: the model (KC1), the system instructions, the tool/integration set (KC5), the memory

architecture and retrieval sources (KC4), the orchestration and workflow logic (KC2, KC3), the permission and identity scopes, and the deployment/environment controls (KC6). A change to any of them triggers the partial replay described in 7.0.5.

- **Harness** - the collection of behavior-defining elements *other than* the model (KC1). It determines which decision points the model encounters and what context each decision sees; the model's outputs in turn determine the harness's next state. The deployed application is the joint product of model and harness, which is why the *trajectory* - not any single decision - is the unit that reflects the system as built (7.2.2).
- **Configuration constraint** - the specific value assigned to a behavior-defining element at design time (which tools are enabled, which memory scope is permitted, which permission boundary is set). Intended scope states what the application should do; configuration constraints are how the design enforces it.

Note on evidence (no new KC). The source proposal introduced a "KC7 - Behavioral Observation & Evidence" with a *behavioral tuple* as the atomic unit of observation (a structured record of one agent action capturing agent identity, session context, triggering input, reasoning step, action taken, state change, active policy context, and timestamp). Per this guide's no-new-KC rule, that is kept as an **evidence/observability concept** rather than a key component: the behavioral tuple is the structured audit record produced by the logging and hook instrumentation already specified in *3.3.3* and *3.3.7*, and it is the foundation for the custom benchmarks in 7.2.1.

7.0.3 The assurance case

An application reaches an acceptable assurance state when its intended scope is explicitly defined, its design constraints are documented, its main risk paths are understood, its controls are linked to tests that can validate or challenge those assumptions, and its known limitations and residual risks are documented and accepted by an accountable owner. The **assurance case** is the combination of:

- **Declared design assumptions** - the scope, trust boundaries, capability constraints, and threat model the system was designed against;
- **Validation evidence** - structured, traceable proof those assumptions were tested under conditions that could falsify them;
- **Runtime evidence** - continuous, queryable proof that the controls encoding those assumptions remain live and correctly operating in production;
- **Change traceability** - a record of how the assurance case was updated when the system changed.

Assurance is the connective layer between the rest of the guide: architecture defines the boundaries, developer controls implement them, capability controls constrain high-risk actions, supply-chain controls preserve the integrity of behavior-defining artifacts, deployment controls enforce runtime limits, and monitoring detects drift. Section 7 asks whether the complete system still behaves within the scope it was

designed to satisfy. This chapter does **not** prescribe red-teaming playbooks, general model-evaluation methods, accuracy benchmarks, compliance mappings, or governance processes.

7.0.4 Design-time assurance scoping

Before any testing begins, the design must produce explicit, testable statements of what the system should do, what it must not do, and under what conditions those constraints hold - these are the targets red teaming and behavioral validation will try to falsify. A complete design-time assurance scope contains:

- **Intended capability boundaries** - for each operational capability exposed through KC5 and reachable in the operational environment (KC6), state explicitly what the agent is permitted to do and what it is not.
- **Trust boundary declarations** - for each input source (user prompts, tool results, memory retrievals, inter-agent messages, web content), declare the trust level assigned and the controls applied before it influences reasoning or action.
- **Threat assumptions** - state which ASI T&M threats the design mitigates, which it accepts as residual risk, and which are out of scope; this is the reference for red-team prioritization.
- **Safe failure states** - for each high-risk capability, define what the agent does when a control fires a DENY/ESCALATE, when a tool call fails, or when a plan produces output that cannot be validated.
- **Operational-environment assumptions** - state any assumptions about the deployment environment that, if violated, would invalidate the controls.

These statements become the living reference for every assurance activity, versioned alongside the codebase. When the system changes, the scope is updated *first*.

7.0.5 Design-to-validation handoff and revalidation triggers

Red teaming and behavioral validation are not independent activities applied after design - they are the means by which the design is challenged. The handoff is a contract:

- **Design** defines the intended boundary and expected in-scope behavior.
- **Threat modeling** identifies how that boundary may fail.
- **Controls** are selected to prevent, reduce, detect, or contain those failures.
- **Red teaming** attempts to falsify the assumptions behind the controls.
- **Behavioral validation** checks the application remains useful within scope - performing authorized work reliably without expanding beyond its boundaries or defensively blocking legitimate in-scope behavior.
- **Findings** are converted into remediation, regression tests, monitoring signals, or design updates.

If a design assumption cannot be linked to a control, test, or monitoring signal, it remains an unsupported assumption rather than part of the assurance case. The following changes to a deployed system must trigger

explicit revalidation of affected scope elements (a *partial replay* of the affected design checks, red-team scenarios, and behavioral cases - not necessarily a full retest):

- Any new tool integration or expansion of KC6 operational capabilities;
- Any change to memory architecture (a KC4 type change);
- Any modification to trust-boundary assignments for input sources;
- Any update to system prompts or agent instructions;
- Any change to orchestration logic or workflow transitions (KC2);
- Any infrastructure change affecting environment assumptions;
- Any incident or anomaly suggesting a declared mitigation did not operate as designed;
- Background catalog drift: tools or skills appearing, disappearing, or being renamed in the MCP catalog without explicit human intervention;
- Changes to third-party declarations relied upon by the assurance case (e.g., tool effect annotations, agent card claims, or skill manifest permissions), including declarations that have aged past their evidence window.

Document the scoping decision and its rationale. **An undocumented decision to skip revalidation is itself an assurance gap.**

7.1 Red Teaming Agentic Applications

Why agentic systems require red teaming. Three properties make them hard to validate by design-time review alone. They are **non-deterministic** - the same input may not reliably produce the same output, so a passing test is one sample of possible behavior (one industry study ran a temperature-zero prompt 1,000 times and saw 80 distinct completions). Agent behavior and effective capability **emerge at runtime** - tool use, memory, autonomy, external data, and accumulated state create execution paths that only appear in context. And agent failures have **real-world consequences** - when an agent can invoke tools, access data, or act with delegated privileges, a failure may be an unauthorized *action*, not merely a wrong answer. Red teaming should be integrated across the lifecycle (pre-deployment, on material change to model/prompt/tool surface/data access/permission boundary, and on a regular cadence between changes), not treated as a one-time launch gate. For execution guidance - specific attack techniques, prompts, tools, and playbooks - refer to the OWASP ASI / CSA Agentic AI Red Teaming Guide and the OWASP GenAI Red Teaming Guide rather than this section.

Red teaming simulates adversarial attacks to detect and address vulnerabilities in different scopes, starting with a specific application through the entire organization's security infrastructure.

Red teaming strategy for agentic applications require complex interactions, autonomous decision-making, and integration with external systems. Proactive red teaming identifies possible attack vectors by assessing the key components of agentic systems, including not only prompt injection techniques, but also privilege escalation through tools, memory poisoning and plan manipulation.

The OWASP ASI Red Teaming Guide for Agentic AI offers a risk-based methodology to handle security concerns with LLMs. It promotes a holistic approach that includes model review, implementation testing, infrastructure assessment, and runtime behavior analysis. Cross-functional coordination amongst data science, cybersecurity, ethical, and legal departments is critical. The guide encourages ongoing monitoring and iterative improvement in response to evolving AI threats.

Prompt injection attacks and red teaming detection strategies:
Prompt injection attacks target vulnerabilities in natural language processing systems by altering input to cause unauthorized actions. Such vulnerabilities might cause sensitive data leakage and bypass of defined guardrails. The use of red teaming tools can mimic multiple prompts to assess susceptibility to these attacks, utilizing technologies such as Promptfoo to automate the detection procedure. Continuous and proactive testing is required to anticipate new prompt injection tactics.

OWASP ASI Red Teaming Guide listing available tools and datasets In Appendix B.

Recently, new frameworks have started to be released and they are targeting agentic AI key components such as tool calling manipulation, privilege escalation and memory poisoning.

Examples of such frameworks:

Tool	Description	URL
AgentDojo	A dynamic evaluation framework to evaluate prompt injection attacks and defenses for LLM agents. Contains targeted injection attacks, instructing the agent to execute malicious tasks using available tools.	https://agentdojo.spylab.ai/
Agentic Radar	Agentic Radar is designed to analyze and assess agentic systems for security and operational insights. It helps developers, researchers, and security professionals understand how agentic systems function and identify potential vulnerabilities.	https://github.com/splx-ai/agentic-radar
Agent-SafetyBench	An evaluation benchmark built primarily to assess the safety of Large Language Model (LLM) agents. Comprises a set of prompts or situations designed to evaluate whether agents follow safety norms and prevent creating damaging, unethical, or risky content or actions.	https://github.com/thu-coai/Agent-SafetyBench
AgentFence	Red team tool equipped with multiple attack scenarios focusing on identifying vulnerabilities like prompt injection, role confusion, and system instruction leakage.	https://github.com/agentfence/agentfence
Agent Security Bench (ASB)	Red team tool equipped with multiple attack scenarios including Direct Prompt Injections (DPI), Indirect Prompt Injections (IPI), Plan-of-Thought (PoT) Backdoor, and Memory Poisoning Attacks.	https://github.com/agiresearch/ASB
Multilingual Benchmark for Agent Security (MAPS)	Security evaluation benchmark built to assess multi-lingual security of AI agent based on Multi-lingual adaptation of set of well known agentic benchmarks (like ASB) translated to 10 different languages.	https://huggingface.co/datasets/Fujitsu-FRE/MAPS

AgentPoison	Red teaming tool used for injecting malicious instances into the knowledge base or memory, to be retrieved when specific triggers appear in the input query.	https://arxiv.org/pdf/2407.12784

Privilege Escalation in Agentic Applications and Red Team Methodologies

Privilege escalation attacks require acquiring illegal access to higher levels of control in a system. The ability of agentic AI to make independent decisions broadens the scope of potential attacks. Red teaming simulates situations to discover privilege escalation threats, such as leveraging inter-agent relationships and manipulating external integrations. Privilege escalation can affect widely, from the application itself, the deployed infrastructure, to affecting the agent's connected agencies, which allow it access to the external environment and additional systems.

Memory Poisoning Threats and Red Teaming Techniques for Identification:

Memory poisoning attacks can use either short-term memory, such as poisoned contextual information injected into the prompt, or through long-term memory and knowledge bases that introduce poisoned content that influence agent's decision-making. AgentPoison and ASB are red-teaming frameworks that employ memory poisoning attacks to discover such vulnerabilities, resulting in a high possibility of retrieving poisoned data.

7.2 Behavioral Testing for Agentic Applications

Security framing. From a security perspective, behavioral testing measures whether the agent remains within its intended scope across representative in-scope usage, complementing the adversarial probing of the *same* boundary done by red teaming. It surfaces failures that require **no attacker**:

1. **Scope drift** - gradual departure from the intended boundary on routine inputs;
2. **Silent composition** - sequences of individually permitted actions that compose into an out-of-scope outcome;
3. **Encoding gaps** - declared constraints never correctly translated into operational controls;
4. **Over-restriction** - blocking or escalation of legitimate in-scope requests.
5. **Control miscalibration** — a binding guardrail, classifier, or policy control incorrectly permits out-of-scope behavior or blocks legitimate in-scope behavior because of inaccurate or poorly calibrated decisions

An agent that over-refuses has failed in its intended scope as surely as one that oversteps it. Persistent refusal may not be a sign of over-caution but an encoding failure or an inaccurate or miscalibrated control. Behavioral testing thereby makes the security–utility trade-off explicit and measurable. Its output is not a score but evidence: observed behavior categorized as in-scope, under-restrictive, or over-restrictive; each finding traceable to a stated design assumption, and, where possible, its root control failure.

Behavioral testing for AI agents emphasizes evaluating their activities and interactions rather than solely examining their underlying structure or code. It is an essential assurance approach to ensure that agentic applications yield consistent and safe outcomes across diverse inputs and scenarios.

This testing approach aims to uncover hidden problems such as unexpected actions, logical errors, or the pursuit of harmful goals. Behavioral testing methods require a systematic approach that encompasses defining explicit objectives for the agent, utilizing benchmark datasets to determine performance metrics, executing simulations in regulated environments to analyze the agent's behavior under varying conditions, and employing both human and automated assessments to evaluate the quality and safety of the results.

- **Benchmarks:** Behavioral testing depends on recognized benchmark datasets and task suites as fundamental elements for standardizing performance evaluation. These benchmarks offer various tasks, datasets, and metrics intended to examine the capabilities and behaviors of AI agents in varied contexts and operational environments.
  Examples:
  - **AgentBench:** A comprehensive benchmark specifically designed to evaluate the task-solving abilities of agents across various domains, including operating systems, databases, and knowledge graphs. It assesses planning, decision-making, and tool usage behaviors.

  - **HELM (Stanford):** HELM evaluates models across multiple metrics including accuracy, robustness, fairness, bias, and toxicity, using standardized scenarios. Its results provide insights into model behaviors related to truthfulness and potential harms.
  - **WebArena & The Agent Company:** This benchmark evaluates autonomous agents performing tasks within realistic, complex web environments, testing behaviors like navigation, form filling, information retrieval, and adapting to dynamic website changes.
- **Evaluation Framework:** A structured assessment framework offers a systematic process for implementing benchmarks and performing testing. These frameworks streamline the process of defining objectives, selecting test cases (often derived from benchmarks), executing tests in controlled or simulated environments, and analyzing results against expected behavioral standards (both positive and negative).

  Examples:
  - **GenAI evaluation service (Google):** Evaluation framework for any generative model or application and benchmark the evaluation results against defined judgment, using the user's evaluation criteria.
  - **Bedrock Evaluations (Amazon):** Evaluation framework for LLMs and integration with external sources as RAGs.

Limits of general-purpose benchmarks. General-purpose benchmarks (AgentBench, HELM, WebArena, and similar) give a useful baseline for broad capabilities but are **not** designed to validate a *deployed* agentic system and should not serve as primary assurance evidence. Three limits apply: (1) **generic context** - they don't evaluate the deployment's specific capability set, trust boundaries, or operational environment, so a high-scoring agent may still over-refuse legitimate requests or under-refuse against its declared boundaries; (2) **no capability composition** - they evaluate capabilities in isolation, not as the deployed capability set, and don't model the lethal trifecta (*2.7.3*); (3) **scores, not evidence** - a score is not traceable to a declared design assumption and can't feed the verification queries below. Their appropriate use is pre-deployment sanity checks, regression detection after model/prompt changes, and a starting template for custom benchmarks (7.2.1). State these limits explicitly when reporting results.

7.2.1 Custom Benchmark Construction

A custom benchmark answers what a general-purpose one cannot: is this agent *refusing requests it should permit*? *Permitting actions it should block*? Does it behave correctly at the boundaries of its declared capability scope?

a) **Action-based construction.** Start from the structured audit record (the behavioral tuple, 7.0.2): the tuple provides the observation structure, the benchmark provides the classification of whether an observed behavior was correct, incorrect, over-restrictive, or under-restrictive for *this* deployment.

1. **Sample from the design-time scope** (7.0.4) - generate test cases from the capability boundaries, trust-boundary declarations, and threat assumptions. Each case targets a specific declared boundary (a permitted capability, a prohibited one, an untrusted input source, a safe failure state).
2. **Classify expected tuples** - for each case, define the expected action_taken and state_delta for correct behavior, and what they'd look like under over-/under-refusal. This is the benchmark specification.
3. **Execute and compare** - run cases against the deployed agent, collect actual tuples, and compare field by field (not just the final output, but the reasoning step, policy context, and state delta).
4. **Classify failures by type** - **security failures** (permitted something it should have blocked / under-refusal vs. declared boundaries), **usability failures** (blocked something it should have permitted / over-refusal vs. legitimate requests), and **instrumentation failures** (the tuple is incomplete - a coverage gap, not an agent-behavior problem). Both security and usability failures are assurance concerns; an over-refusal is often a signal that capability boundaries weren't correctly encoded into the operational controls.
5. **Integrate into CI/CD** - a change to behavior-defining elements that shifts behavior relative to declared boundaries should block release the same way a failing security test does (see the revalidation triggers in 7.0.5).

b) **Trajectory-based construction.** Classifying a single action tests instruction-following at one decision; it does not test whether the *path* stayed in scope. Two security-relevant properties are invisible at the single-action level: **harness steering** (the path is the joint product of model and harness, each conditioning the other) and **composition** (a sequence of individually permitted actions can compose into an out-of-scope outcome - the lethal trifecta is a property of a *path*, not any one action). A trajectory is the ordered sequence of steps (reasoning, tool selection, memory read/write, output) with each resulting state change and active policy context; a trajectory benchmark classifies whether that *sequence* stayed within the intended scope. The unit of evaluation is the path, not the endpoint. Four deviation types, each needing a different detection frame and response:

Deviation type	Where	Trajectory signature	Benchmark assertion	Detection frame
Honest misexecution	In-session	Path wanders off task, incoherent toward any goal, stalls or self-corrects	Trajectory completes the intended task without leaving scope	Declared scope & task envelope

Overreach	In-session	Goal-directed sequence of individually permitted steps that incrementally extends capability, data access, or egress past the boundary	No step sequence crosses a scope boundary, even where each step is allowed	Declared scope, evaluated over the sequence
Prohibited composition	In-session, usually adversarially induced	Benign steps that jointly satisfy a prohibited composition (e.g., private-data read → untrusted content → external egress)	Prohibited compositions never co-occur on a path	Declared prohibited compositions (*2.7.3*)
Cross-session drift	Across sessions	Step distribution diverges from the in-scope baseline (new tool patterns, rising frequency at scope edges)	Indirect: passing trajectories seed and recalibrate the baseline; drift is detected in production monitoring as divergence from it	Empirical baseline & declared scope

Detection uses **two reference frames**: *declared scope* (normative - what is allowed; the frame for misexecution, overreach, and malicious composition, all assessed by executing benchmark scenarios) and *empirical baseline* (descriptive - what is typical, aggregated from in-scope trajectories across sessions; the frame for cross-session drift, detected in production monitoring). They are complementary: a trajectory may be in-scope but atypical (emerging drift) or typical but newly out-of-scope (the scope or its controls shifted). Construct and run trajectory scenarios with the same five-step process lifted to the path level - sample scenarios that each include at least one benign-looking composition that should be prohibited; define the expected in-scope envelope plus labeled deviating sequences; execute and compare at the trajectory level; classify by deviation type (plus instrumentation failures); and integrate into CI/CD so passing trajectories seed the cross-session baseline.

For Agentic Application security, the first step in selecting an appropriate benchmark is determining the type of benchmark required.

Steps to take to make the right choice:
AI security benchmarks today differ widely in scope, evaluation methods, and threat coverage, leading to gaps in defense and inconsistent comparisons. The OWASP GenAI Red Teaming Guide has already curated a list of AI security benchmarks, focusing on key threat vectors. Given the diversity and specificity of these benchmarks, a step-by-step guideline is necessary to help organizations effectively choose and apply the right benchmark for their unique security needs. This structured approach ensures that AI systems are tested in a comprehensive and reproducible manner, leveraging the curated benchmarks without redundancy, while addressing emerging vulnerabilities in a consistent, actionable way.

Anthropic's initiative for third-party model evaluations highlights that not all benchmarks are focused on AI security. Some assess general performance, like accuracy, rather than security vulnerabilities. Understanding the specific evaluation goal is crucial before selecting a benchmark. By identifying whether the focus is on performance, security, or robustness, organizations can choose the most relevant benchmarks, ensuring they align with the intended objectives and test AI systems for the right aspects.

The EU AI Act's Code of Practice also emphasizes safety and security measures for AI systems, aiming to mitigate risks associated with their deployment. AI security focuses on protecting systems from malicious threats, while AI safety ensures that systems operate as intended without causing unintended harm.

AI security benchmarks need meta-evaluation to uncover issues. Examples are inconsistency (variance introduced), ambiguity (benchmark misunderstood), incompleteness (missing user needs), or inaccuracy (bias introduced). This ensures benchmarks stay relevant, realistic, and genuinely reflect AI Application security.

By embedding lifecycle practices-scheduled updates, community peer review, and governance checks-the guideline keeps benchmarks up to date as adversarial techniques evolve. This structured, security-centric process ensures organizations select benchmarks that truly reflect and defend against the current AI threat landscape.

Step-by-Step Guide: Selecting a Security Benchmark for Agentic AI Applications

1. **Define Security Objectives** - Identify your security goals based on AI security principles:
    - Confidentiality: Protect training data and model parameters.
    - Integrity: Ensure data and model integrity.
    - Availability: Keep services operational.
    - Adversarial Robustness: Protect models from adversarial attacks.
    - Privacy Protection: Use techniques like anonymization.
    - Transparency and Accountability: Ensure explainability and traceability.

- Governance and Compliance: Follow frameworks like NIST RMF.

2. **Assess Threat Landscape** - Identify relevant threats, such as:
    - Data poisoning.
    - Adversarial attacks.
    - Model extraction.
    - Privacy breaches.
    - Denial-of-Service (DoS).
    - Misuse of agent autonomy.

3. **Research Existing Benchmarks** - Look for benchmarks addressing agentic AI vulnerabilities, such as:
    - Adversarial robustness.
    - Privacy protection.
    - Backdoor detection.
    - Model extraction security.

4. **Evaluate Benchmark Criteria** - Ensure the benchmarks:
    - Align with your security objectives.
    - Cover the relevant threats.
    - Are standardized and widely accepted.
    - Provide actionable insights.

5. **Test Benchmark Suitability** - Test benchmarks on representative models or datasets. Focus on:
    - False positives.
    - Effectiveness under adversarial conditions.
    - Resilience under attack.

6. **Compare and Select** - After testing, compare benchmarks based on coverage, relevance, and effectiveness. Select the one that best fits your needs.

7. **Establish Continuous Evaluation** - Implement continuous evaluation to adapt to new threats:
    - Regularly re-test with updated benchmarks.
    - Integrate benchmarks into CI/CD pipelines.
    - Adjust the threat model as new risks emerge.

8. **Engage with the AI Security Community** - Stay informed by participating in forums, workshops, and discussions. Engaging with the community provides insights on the latest security and benchmark developments.

Explainability Audits

- Use CoT for some insight into agentic reasoning, understanding that what the agent says and "thinks" aren't necessarily aligned. (*https://transformer-circuits.pub/2025/attribution-graphs/biology.html*)
- Utilize HITL at critical junctions to understand and gate agent actions. Criticality is defined by regulations like the EU AI Act.
- Use explainable AI (XAI) strategies as appropriate to the model in use: SHAP/LIME for structured decision models; Grad-CAM only if the agent has a vision component (see multimodal note above); t-SNE for auditing memory/RAG embedding clusters rather than reasoning traces generally.
- Record agent to agent goal negotiations and tool interactions to understand additional inputs which may be misdirecting an agent. Especially for the more sensitive exchanges/tool calls, like database modifications.
- Log which permission sets or identities were used for agentic actions. This will indicate who the agent "thought" it was acting on behalf of.

8. Secure Agentic Deployments

Deploying LLM agents into production environments requires careful consideration of security beyond the model's inherent capabilities. While the specifics depend heavily on the application and risk profile, several principles and practices contribute to a more robust and secure agentic system. The following points provide a flavor of key areas to address:

- **Secure Pipelines and Checks for Rogue Agents:** Implementing rigorous CI/CD pipelines is important, but agent security requires more than standard code checks. These pipelines must incorporate automated vulnerability scanning of dependencies, static analysis focusing on potential insecure Agent's tool usage patterns, and dynamic testing specifically designed to probe for prompt injection vulnerabilities or unintended goal deviation before deployment. Furthermore, incorporating code signing, provenance tracking for agent versions, and potentially manual security reviews for agents handling high-risk tasks helps ensure that only vetted and approved agent logic reaches production, preventing deliberately malicious or inadvertently vulnerable agents from being deployed.
- **Role Containerization or Function-as-a-Service (FaaS):** To limit the potential blast radius should an agent instance be compromised, each agent or distinct agent role should operate within a strongly isolated environment. Utilizing containerization technologies (like Docker with Kubernetes) or FaaS platforms (like AWS Lambda or Google Cloud Functions) allows for strict resource limits (CPU, memory, network), network segmentation policies, and ephemeral execution contexts. This enforces the Principle of Least Privilege at the infrastructure level, preventing a compromised agent from easily accessing resources or impacting other agents or system components beyond its designated scope.
- **API Access Control, Rate-Limits, and API Gateways:** Agents heavily rely on external tools accessed via APIs, or other integration/interaction points such as MCP Server or Google A2A Agent Server via AgentCard, making these interaction points critical security boundaries. An API/Agent Gateway should mediate all tool invocations, enforcing strict authentication (verifying the agent's identity) and authorization (confirming the agent has permission to call that specific tool with those parameters). Implementing granular rate limiting per agent or per tool prevents denial-of-service attacks (intentional or accidental) and controls costs associated with API usage. The gateway also serves as a vital point for logging and auditing all tool interactions.
- **Runtime Security with Memory Isolation (e.g. CaMeL - Capabilities for Machine Learning by Google):** Incorporating runtime security architectures like Google's CaMeL provides defense-in-depth even if the underlying LLM is vulnerable, for instance to prompt injection. By explicitly

separating control flow generation (Privileged LLM seeing only trusted prompts) from untrusted data processing (Quarantined LLM with no tool access), application developers can use static APIs/methods or a custom interpreter that enforces security policies based on fine-grained capabilities and data flow tracking before executing tool calls. These methods prevent untrusted data from directly influencing the agent's actions or exfiltrating sensitive information via unauthorized tool parameters. This empowers security to avoid relying solely on model alignment, to enforce data flow through system design.

- **Separating Data Planes from Control Planes (Multi-Agent Setups):** In systems where multiple agents collaborate, the mechanisms for inter-agent communication must differentiate between control messages (e.g., task assignments, commands) and data messages (e.g., shared information). Control channels require stronger authentication, authorization, and integrity checks to prevent a compromised agent, potentially handling untrusted data, from issuing malicious commands or disrupting the coordination of the entire system. This separation limits the ability of a single agent compromise to cascade into a systemic failure.
- **Just-in-Time (JIT) Access and Ephemeral Credentials:** Agents requiring access to sensitive tools or data should operate under the principle of least privilege in time. Instead of using long-lived static credentials for tool invocations, employ mechanisms that grant access permissions only when needed and for the shortest duration possible (JIT). This should be coupled with the use of ephemeral credentials, such as short-lived API tokens (e.g., OAuth tokens, signed JWTs) or temporary cloud credentials (e.g., AWS STS, GCP IAM credentials), which expire automatically, reducing the window of opportunity for misuse if they are compromised.
- **Alerting on Anomalous Behavior:** Continuous monitoring and alerting are essential for detecting potential compromises or malfunctions. Systems should baseline normal agent behavior (e.g., typical tool usage frequency and sequences, resource consumption) and trigger alerts upon significant deviations. Particular attention should be paid to unexpected or unauthorized tool use attempts, sudden shifts in the agent's apparent goal or task execution logic (which might indicate successful injection or manipulation), or anomalous patterns in data access or external communication, enabling rapid investigation and response.
- **Human Oversight in High-Stakes Environments:** For agents operating in domains with significant consequences (e.g., finance, healthcare, education, or critical infrastructure), full autonomy is often inappropriate. Implement robust human-in-the-loop workflows where critical decisions, high-risk actions (like large financial transfers or sharing sensitive data), or significant deviations from expected behavior require explicit human review and approval before execution. Adaptive trust mechanisms can dynamically adjust the level of required oversight based on the agent's performance, context, and the risk associated with the specific action. For a more complete definition of high-risk systems, review the EU AI Act's Annex on high-risk systems.
- **Managing Non-human Identities:** Each agent instance or service requires a distinct, manageable identity for secure interaction with APIs, tools, other agents and other system components. These machine identities (e.g., service accounts, workload identities) must be treated with the same rigor

as human identities, involving secure provisioning processes, robust credential management (including secure storage and regular rotation of keys/tokens using secrets managers), clear ownership, and reliable de-provisioning when an agent is retired to prevent orphaned, potentially exploitable identities.

9. Securing Agentic Applications across different runtimes & deployment Environments

> **Numbering note.** In V1 the deployment material spanned two headings - "8. Secure Agentic Deployments" (the prose principles, now in *Section 8*) and a runtime-hardening block whose subsections were numbered 9.1–9.6 under a chapter still labeled "8" in the table of contents. This guide resolves that bug by keeping the runtime-hardening block as **Section 9** with subsections 9.1–9.7. Section 8 holds the deployment principles; Section 9 holds the concrete VM/runtime hardening.

Securing agentic applications running in a VM means combining traditional VM security hardening with agentic-specific controls like sandboxing, auditability, capability constraints, and runtime behavioral monitoring. Here's a breakdown tailored for LLM-driven or tool-augmented agentic apps:

9.1 Harden the Virtual Machine (Base Level)

Before even talking agentic specifics, the host VM must be secure:

Baseline VM Hardening

- Use minimal base images (e.g., distroless, Alpine, microVMs).
- Enable Secure Boot, TPM-backed disk encryption (e.g., via dm-crypt/LUKS).
- Disable all unnecessary services (sshd, cron, etc.).
- Apply firewall rules (iptables/nftables) to restrict ingress/egress by default.
- Patch regularly using automation (e.g., unattended upgrades or distroless image re-pulls).

Network Isolation

- Put the VM inside a private subnet or VPC.
- Use Service Meshes or API Gateways for controlled inter-agent communication.
- Prevent direct internet access unless explicitly needed (and monitor it).

9.2 Contain the Agentic Runtime

Now, focus on the agentic app itself running in the VM:

Sandbox the Agent Processes

- Run each agent or tool in its own restricted namespace or container inside the VM (e.g., via gVisor, Firecracker, or even Docker-in-VM).
- Use AppArmor/SELinux or seccomp profiles to restrict syscalls.
- Limit file system access using mount namespaces and read-only volumes.
- Inject ephemeral storage (e.g., tmpfs) for memory-sensitive LLM tools.

Scoped Capabilities

- Use an AGNTCY-like capability control layer:
    - Agents can only use explicitly declared tools (e.g., searchDocs, not shellExec).
    - Introduce capability_tokens scoped per session.
    - Use JSON-RPC wrappers (like MCP) to enforce capability boundaries.

9.3 Secure the Agent's Memory, Tools, and Context

Memory and State Hygiene

- Encrypt in-memory state (where feasible).
- Auto-clear memory on session end (e.g., clear Redis/Vector DB agents).
- Use VM hooks (e.g., systemd unit stop triggers) to flush memory or rotate keys.

Tool Execution Control

- Implement runtime guards for tool calls:
    - Validate inputs and outputs.
    - Scan for prompt injection or malicious redirection attempts.
- Use AGNTCY's policy engine to gate access dynamically:
    - E.g., only allow call_api("billing") from the "finance-agent."

9.4 Observability + Forensics

Audit Everything

- Every agent action (tool use, messages, memory write) should:
  - Be logged with timestamp, agentID, sessionID, payload hash.
  - Include input/output signature fingerprints.
- Store logs securely with forward integrity (e.g., append-only + Merkle proofs or signed logs).

Behavioral Anomaly Detection

- Use runtime policy engines (e.g., Open Policy Agent) or integrate LLM anomaly detectors to spot:
  - Unusual tool invocation patterns.
  - Cross-agent communication deviations.
  - Memory bloat or frequent memory peeks.

9.5 Identity, Authentication, and Agent Authorization

Identity & Session Management

- Use JWTs, mTLS, or HMAC-based auth per agent session.
- Assign each agent a unique ephemeral session context using MCP.

Role-based + Capability-based Access

- Combine RBAC (agent classes) with capability tokens.
- Example:
  - research-agent → can access docSearch, summarize.
  - ops-agent → can use issueCommand, getStatus.

9.6 (Optional) Cloud-Specific Hardening

If your VM is running in AWS/GCP/Azure:

- Restrict metadata service access (e.g., IMDSv2 in AWS).
- Use IAM roles scoped per VM, not global.
- Enable Confidential Computing if supported (e.g., AMD SEV, Intel SGX).

9.7 Runtime Enforcement Additions

These controls operationalize, at deployment time, the design-time concepts introduced earlier in the guide. They are deterministic enforcement points - not prompt-level mitigations.

Deterministic sandboxing vs. probabilistic safety

Distinguish deterministic boundaries (OS isolation, policy proxies, cryptographic checks) from probabilistic, prompt-based safety. The former are primary security controls; the latter are best-effort. Use gVisor/Firecracker for kernel isolation, eBPF for fine-grained syscall monitoring alongside AppArmor/SELinux/seccomp, and network/path isolation (see *3.2.3*).

Execution integrity

Authorization does not guarantee the executed request equals the approved request. Enforce authorized_request == executed_request: the selected tool, target, parameters, and identity must remain unchanged from model intent to execution - via a signed execution artifact (hash of the request payload), mandatory mediation through a tool proxy that re-verifies at execution time, and replay protection on authorization artifacts.

Tool gateway / MCP proxy

Mediate all tool invocations through an API/Agent gateway or MCP proxy/sidecar; direct agent-to-server connections are prohibited in production. The gateway enforces server-identity authentication, request/response schema validation, per-tool and per-session rate limiting, least-privilege catalog scoping, and tool/server allowlisting at the proxy (not the system prompt). See the tool-lifecycle controls in *Section 6*.

Memory provenance & trust metadata

Persist a *Standardized Provenance Record (6.4.1)* with every memory record to track author, trust level, and integrity. Integrity proves a record was not modified; it does not prove the content is safe - pair it with semantic validation and trust labeling so retrieved memory is re-checked before it can influence reasoning or action (see *3.1.6*). Segment memory by tenant/user/workflow and apply TTL/expiration.

Hooks as lifecycle enforcement points

Wire the hook points from *3.3.7* into the runtime so that input, tool-call request/response, memory store/retrieval, and final-output enforcement run synchronously and fail closed. Where the framework's native hooks are observe-only, enforce through synchronous wrappers.

Behavioral circuit breakers

Deploy the circuit breakers from *3.3.8* as in-line, blocking runtime policy engines (e.g., Open Policy Agent) with graduated tiers (warn → throttle → suspend → kill). Enforcement is decentralized for speed; policy management can be centralized for coherence.

Incident snapshots before kill/rollback

Before a circuit breaker kills or rolls back an agent, capture an incident snapshot: freeze the context window and preserve the tamper-evident provenance chain (principals, decisions, tools, data accessed), using hash-chaining of harness events and signed export bundles so the audit record is a verifiable proof rather than a trust assertion.

Acknowledgements

Contributors

Aamir Syed
Aamiruddin Syed
Ambuj Kumar
Andrea Siposova
Angelika Szymanowska
Ariel Fogel
Bar Kaduri
Brian M. Green
Dipen Shah
Emile Delcourt
Francesco
Gautam Bharti
Guy Shtar
gyanendar manohar
Harel Gal
Hari Holla
Hariprasad Holla
Idan Habler
Jeff Ponte
Joan Ekaterin
Joshua Beck
Joshua Lochner
Karmvir Jadeja
Kaustubh Phatak
Krishna Puranik
Markus Hupfauer
Mohammed Ilyas Ahmed
Nir Dagan
Or Oxenberg
Eliran Suisa
Rani Kumar Rajah
Recep Meric Degirmenci
Rock Lambros
Sahil Mehta
Sanjeev Agarwal
Shaked Reiner
Shoval Abudraham
Sita Ram
Sita Ram Sai
Sriram Prasath Narayanan
Sumer Jagtap
Sushma Mahadevaswamy
Tal Dromi
Tom Barnea
Tom Stripling
Venkat Ragothaman
Volkan Kutal
Yuval Sarel

OWASP GenAI Security Project Sponsors

We appreciate our Project Sponsors, funding contributions to help support the objectives of the project and help to cover operational and outreach costs augmenting the resources provided by the OWASP.org foundation. The OWASP GenAI Security Project continues to maintain a vendor neutral and unbiased approach. Sponsors do not receive special governance considerations as part of their support.

Sponsors do receive recognition for their contributions in our materials and web properties. All materials the project generates are community developed, driven and released under open source and creative commons licenses. For more information on becoming a sponsor, visit the Sponsorship Section on our Website to learn more about helping to sustain the project through sponsorship.

Project Sponsors:

GenAI Security Project – Gold Project Sponsor – Top 10 for LLM and Generative AI

WITNESS AI · HIDDENLAYER · FUJITSU · paloalto NETWORKS · Straiker · Mend.io · snyk · f5 · ByteDance · TREND MICRO

GenAI Security Project – Silver Project Sponsor – Top 10 for LLM and Generative AI

ALICE · GT cyber · zscaler · evoke SECURITY · CROWDSTRIKE · PROTECTO · SentinelOne · PROMPTARMOR · proofpoint. · Cobalt · securiti · apiiro · Capsule · Synack · akto · Starseer · NeuralTrust · CHECK POINT · TROJ.AI · LASSO

Sponsors list, as of publication date. Find the full sponsor list here.

Project Supporters

Project supporters lend their resources and expertise to support the goals of the project.

Accenture
AddValueMachine Inc
Aeye Security Lab Inc.
AI informatics GmbH
AI Village
aigos
Aon
Aqua Security
Astra Security
AVID
AWARE7 GmbH
AWS
BBVA
Bearer
BeDisruptive
Bit79
Blue Yonder
BroadBand Security, Inc.
BuddoBot
Bugcrowd
Cadea
Check Point
Cisco
Cloud Security Podcast
Cloudflare
Cloudsec.ai
Coalfire
Cobalt
Cohere
Comcast
Complex Technologies
Credal.ai
Databook
DistributedApps.ai
DreadNode
DSI
EPAM
Exabeam
EY Italy
F5
FedEx
Forescout
GE HealthCare
Giskard
GitHub
Google
GuidePoint Security
HackerOne
HADESS
IBM
iFood
IriusRisk
IronCore Labs
IT University Copenhagen
Kainos
KLAVAN
Klavan Security Group
KPMG Germany FS
Kudelski Security
Lakera
Lasso Security
Layerup
Legato
Linkfire
LLM Guard
LOGIC PLUS
MaibornWolff
Mend.io
Microsoft
Modus Create
Nexus
Nightfall AI
Nordic Venture Family
Normalyze
NuBinary
Palo Alto Networks
Palosade
Praetorian
Preamble
Precize
Prompt Security
PromptArmor
Pynt
Quiq
Red Hat
RHITE
SAFE Security
Salesforce
SAP
Securiti
See-Docs & Thenavigo
ServiceTitan
SHI
Smiling Prophet
Snyk
Sourcetoad
Sprinklr
stackArmor
Tietoevry
Trellix
Trustwave SpiderLabs
U Washington
University of Illinois
VE3
WhyLabs
Yahoo
Zenity

Supporters list, as of publication date. Find the full supporter list here.